# Search Engine Project

Stage 2: Service-Oriented Architecture


| | |
|---|---|
| **Course** | Big Data · Grado en Ciencia e Ingeniería de Datos · Universidad de Las Palmas de Gran Canaria |
| **Academic year** | 2026/2027 |
| **Group name** | BigData-Ulpgc |
| **Members** | Daniel Rodríguez Alonso — 79072763M |
| | Daniel Perdomo Medina — 54173402F |
| | Carlos Falcón Brito — 54149163X |
| **Repository** | https://github.com/BigData-Ulpgc/stage_2 |

# Contents

**2 Introduction and objectives** **3**

**3 System architecture** **3**

3.1 Coordination mechanism . . . 3
3.2 Deployment on one host . . . 4
3.3 The life of a book . . . 5

**4 API design and contracts** **7**

4.1 Front-end use cases . . . 7
4.2 Internal and operational interfaces . . . 7
4.3 Parameters, pagination and errors . . . 8
4.4 Contracts that are not HTTP . . . 9
4.5 Validation evidence . . . 9

**5 Microservice implementation** **9**

5.1 Ingestion and Control Module . . . 9
5.1.1 Ingestion Service . . . 10
5.1.2 Control Module . . . 12
5.2 Indexing . . . 13
5.2.1 Readiness, task ownership and retries . . . 13
5.2.2 Idempotency . . . 14
5.2.3 Publication of completed index updates . . . 14
5.2.4 Full rebuild of the index . . . 15
5.3 Search and deployment . . . 16
5.3.1 Search Service . . . 16
5.3.2 Deployment . . . 17

**6 Design decisions** **18**

6.1 Coordination . . . 18
6.1.1 Why Hazelcast . . . 18
6.1.2 How the shared state is protected . . . 18
6.1.3 Single machine, replicas, threads and ports . . . 19
6.1.4 Where replication helps and where it contends . . . 19
6.1.5 Failure tests . . . 20
6.2 Storage and concurrency . . . 21
6.2.1 Storage selection . . . 21
6.2.2 Concurrency model of Indexing . . . 22
6.2.3 Where threads and replicas help, and where they contend . . . 22

**7 Benchmarks and results** **23**
7.1 Ingestion . . . 23
7.1.1 Microbenchmarks with JMH . . . 23
7.2 Indexing . . . 25
7.2.1 Microbenchmarks with JMH . . . 25
7.2.2 Integration experiments . . . 25
7.2.3 Analysis . . . 27
7.3 Search . . . 28
7.3.1 Reference machine . . . 28
7.3.2 Microbenchmarks with JMH . . . 29
7.3.3 Load tests . . . 31
7.3.4 Analysis . . . 35
7.3.5 Search while the pipeline works . . . 36
7.4 Consolidation . . . 37
7.4.1 The host . . . 37
7.4.2 What was measured . . . 38
7.4.3 Bottlenecks and scaling trends . . . 38

**8 Conclusions and evolution to Stage 3** **39**
8.1 Strengths and weaknesses of the coordination . . . 39
8.2 What changes on the Stage 3 cluster . . . 39

## 2. Introduction and objectives

Stage 1 built and measured the data layer of a search engine over the books of Project Gutenberg: a datalake, an inverted index and a metadata store. Stage 2 turns that data layer into a system of cooperating services that keeps working while books are being added and users are searching, all on a single machine.

The system has four kinds of process. **Ingestion** downloads books and stores them in the datalake. **Indexing** adds complete books to the index and publishes new versions of it. **Search** answers the front-end. The **Control Module** decides what has to be done and when, without doing any of the work itself. They coordinate through Hazelcast and share their results through the datalake and MongoDB. A front-end only sees one public API, behind nginx.

The objectives of the stage, and how the system meets them, are:

- **Clear contracts.** Every HTTP endpoint is specified in OpenAPI and classified as front-end, internal or operational, and the interactions that are not HTTP have their own contracts (Section 4).
- **Correct behaviour under concurrency.** A book is only indexed once complete, each book is processed by one worker at a time, and a search never sees a partially published version.
- **Survival of failures.** A worker that dies loses its lease and its book returns to the queue. If Hazelcast loses its state, the Control rebuilds it from what is persistent. If a Search replica dies, nginx sends the request to another one.
- **Reproducible deployment.** One command starts every process with fixed resource limits, and the number of replicas and threads of each service is configurable (Section 5.3.2).
- **Evidence.** The APIs are checked automatically against the running system, and the benchmarks identify what limits performance on one machine, keeping client concurrency, threads and replicas apart (Section 7.3).

Everything runs on one host by design: replicas and threads share its CPU, memory and disk. What these measures say about one machine does not establish how the system will scale on the Stage 3 cluster.

## 3. System architecture

The search engine of Stage 1 becomes four cooperating services, each a Maven module that builds an executable jar and a Docker image: **Ingestion** downloads books into the datalake, **Indexing** turns them into the inverted index and the metadata in MongoDB, **Search** answers the queries of the front-end, and the **Control Module** feeds and supervises the pipeline. They never call each other to hand over work: they share it through a **Hazelcast** cluster, which holds the state of every book and the queues of work. Only two calls go over HTTP between services: the Control asks Indexing to publish, and the administrators call the internal APIs directly.

### 3.1 Coordination mechanism

The design combines two of the mechanisms the assignment allows:

- **A shared-memory region:** the Hazelcast cluster (two members, version 5.7.0). It is the only place where the services meet. It holds the state of every book (`book-state`), the two queues of work (`ingest-queue` and `index-queue`), the lock of the single index

writer (`index-writer`), the event of each new version of the index (`index-events`) and a marker that tells whether the cluster still has its state (`coordination-meta`). All access goes through one library in `common` (`Coordination`), so no service builds a key, a value or a transaction by hand.

- **A scheduler:** the Control Module. It seeds the books, reviews the leases every `LEASE_CHECK_SECONDS` and decides when a new version of the index is published. It is not in the path of any book: the workers take their work from the queues, and keep doing so while the Control is down.

There is no message broker. Hazelcast's queues have no acknowledgements, so a book taken by a worker that dies would be lost from the queue; the leases in `book-state` recover it (Section 5.1.1). The persistent truth is not in Hazelcast but in the datalake and MongoDB: Hazelcast only says who does what, and a book changes state only after its evidence is on disk. If the whole cluster is lost, the Control rebuilds it from that evidence.

## 3.2 Deployment on one host

Every process runs on one machine, started by Docker Compose from `docker-compose.yml` and `.env` (Figure 1). Each container has fixed `cpus` and `mem_limit`, so the experiments are reproducible. The only entry from outside is nginx: it is the **front-end API boundary**, and everything to its right in the figure is internal.

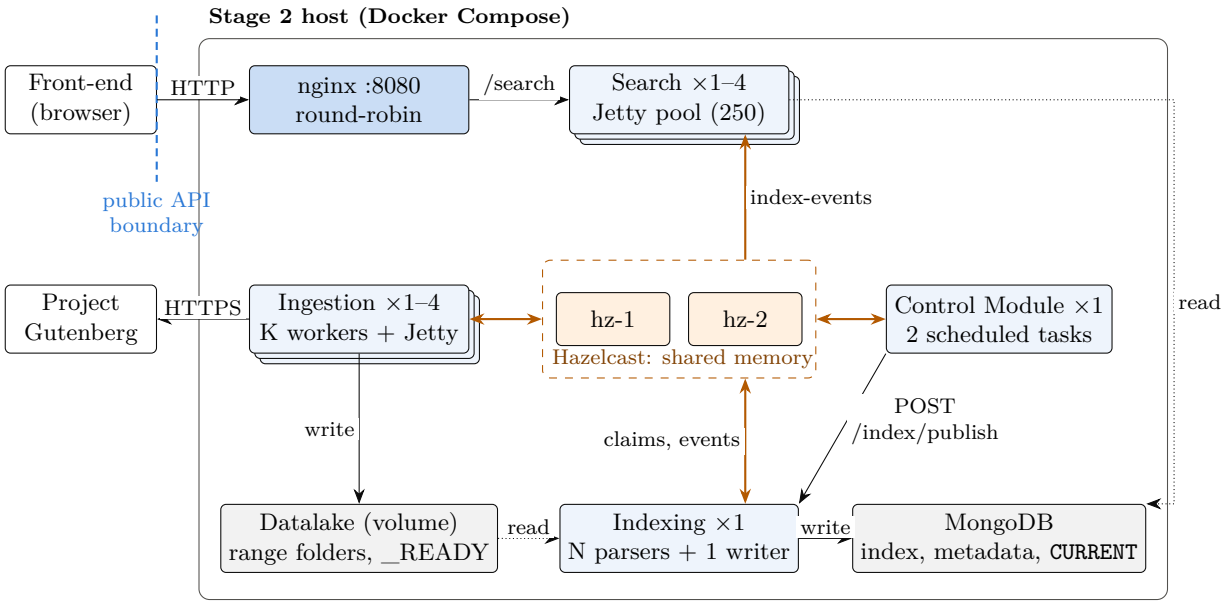



Figure 1: The deployment on the single Stage 2 host. Only nginx is reachable from other machines; the Control (`127.0.0.1:7000`), Hazelcast and MongoDB are published on the loopback interface for the administrator and the tests. Orange: Hazelcast clients (a client reaches both members). Dotted: read-only access to shared storage. A shadow marks a service that can run several replicas.

**Shared storage.** Two stores are shared, each with a single writer:

- The **datalake** is a bind-mounted folder. Ingestion writes it; Indexing and the Control only read it (read-only mounts). Books are grouped in folders of 1,000 ids (`01000-01999`), each with `<id>.header.txt`, `<id>.body.txt` and, written last, `<id>._READY`: a book exists only with its marker (Section 5.1.1).

- **MongoDB** holds the inverted index, the metadata of the books, the log of indexed books (`index_log`), the books of each published version (`index_books`) and the pointer `CURRENT`. Indexing writes; Search reads; the Control reads it only to recover.

**Replicas and threads.** Table 1 separates the three kinds of concurrency of the system: process replicas, threads inside a process, and client concurrency, which only exists in front of Search.

| Process | Replicas | Threads | How work is shared |
|---|---|---|---|
| Ingestion | 1–4 (`--scale`) | `INGESTION_WORKERS` (4) workers; Jetty pool | Every worker of every replica claims from the same `ingest-queue`: Hazelcast distributes the books, and nobody knows the address of a replica |
| Indexing | 1–2 | `INDEXING_THREADS` (4) parsers; one writer | Only the replica that holds `index-writer` claims from `index-queue` and writes the index; another one would wait as a standby |
| Search | 1–4 (`--scale`) | Jetty pool, `SEARCH_HTTP_THREADS` (250) | nginx sends each request to the next replica (round-robin); each one reads an immutable snapshot |
| Control | 1 | 2 scheduled tasks; Jetty pool | Not in the path of a book: lease check and publication |
| Hazelcast | 2 members | managed by Hazelcast | 271 partitions spread over both, each with one backup on the other |
| MongoDB | 1 | managed by MongoDB | Shared by Indexing (writes) and every Search replica (reads) |

Table 1: Replicas and threads of each process. The Jetty pool of every service is explicit: Javalin 6.1.3 creates a `QueuedThreadPool` with at most 250 threads, at least 8 and 60 s of idle time, and `<SERVICE>_HTTP_THREADS` changes the maximum.

### 3.3 The life of a book

`book-state` keeps one entry per book with its state, its owner and its lease, and the transitions are the only way to change it (Figure 2). Each transition is a compare-and-set, `replace(id, expected, next)`, which succeeds only if nobody changed the entry since it was read; one that also moves the id to or from a queue runs in one Hazelcast transaction with that queue operation, so a crash never does one without the other.

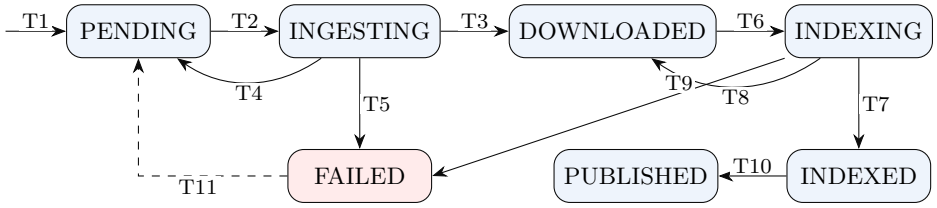



Figure 2: The states of a book (coordination contract §4). T2/T6 claim the book with a lease; T3/T7 complete the stage once its evidence is persistent; T4/T8 requeue it after a transient failure or an expired lease; T5/T9 send it to `FAILED` after a permanent error or the last attempt; T10 marks it visible; T11 is `POST /control/retry-failed`. The recovery (row R, Section 5.1.2) may also create an entry directly as `DOWNLOADED`, `INDEXED` or `PUBLISHED`.

Figure 3 follows one book through the whole system. The order matters: each step writes its evidence before the state that announces it, so a crash between both leaves a book that is either redone or recovered, never one that is announced but missing.

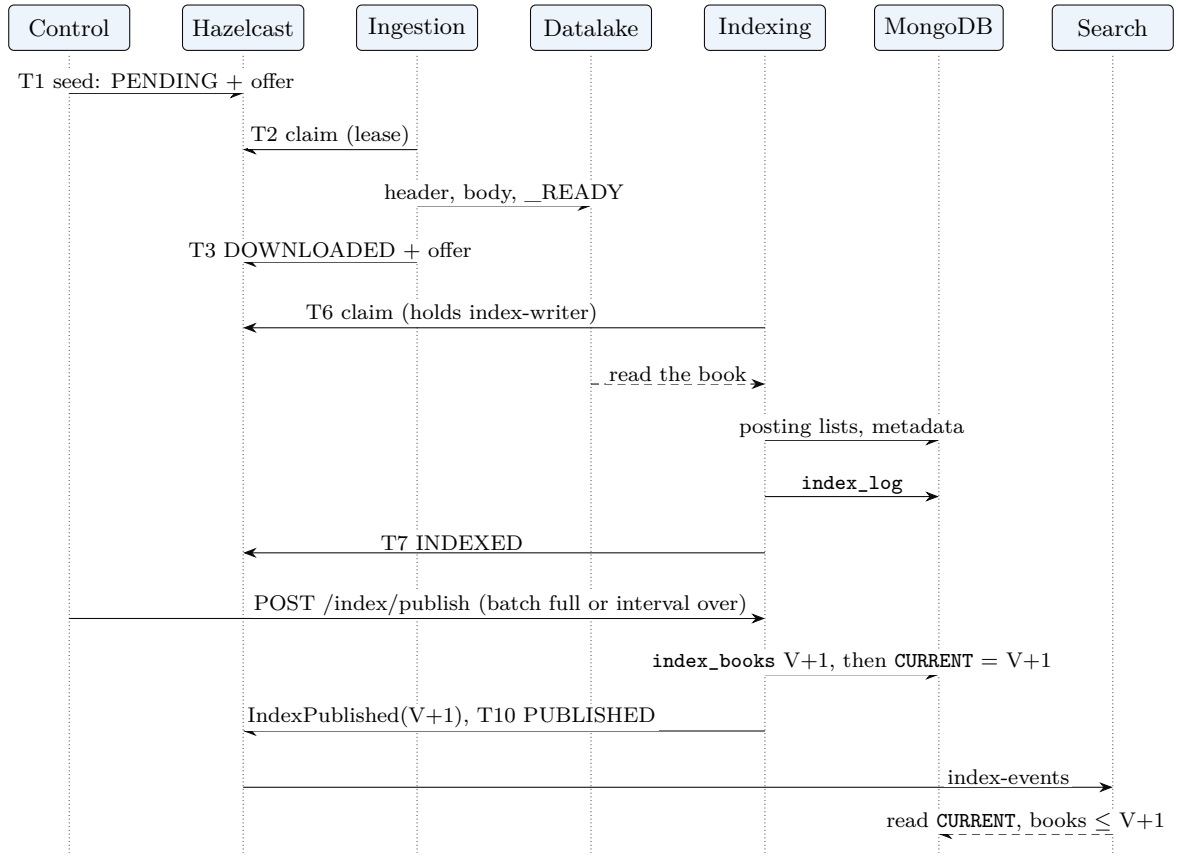



Figure 3: The flow of one book. Dashed: reads. Every arrow towards Hazelcast is a transition of Figure 2; each one comes after the evidence it announces (`_READY`, `index_log`, `CURRENT`).

1. **Seed.** `POST /control/jobs` (or `CONTROL_SEED_FILE`) puts every unknown id as `PENDING` and offers it to `ingest-queue`. A known id is left as it is, so seeding twice duplicates nothing.
2. **Ingestion.** A worker claims the id: one transaction polls the queue and writes `INGESTING` with its name and a lease of `LEASE_SECONDS`. It downloads the book, splits header and body, writes both files and then the `_READY` marker, and completes: `DOWNLOADED` and an offer to `index-queue`, again in one transaction.
3. **Indexing.** The replica that holds `index-writer` claims from `index-queue`, parses and tokenizes the books in parallel, writes the posting lists and the metadata to MongoDB in batches, records each book in `index_log` and marks it `INDEXED`. The book is in the index, but Search does not show it yet.
4. **Publication.** When `PUBLISH_BATCH` books (100) are `INDEXED`, or `PUBLISH_INTERVAL_S` (30 s) have passed with at least one, the Control calls `POST /index/publish`. Indexing records the new books in `index_books` with version V+1 and moves `CURRENT` to V+1 in one atomic update; the index itself is not copied. Right after answering, it announces `IndexPublished` and marks the books `PUBLISHED`.
5. **Reload.** Every Search replica receives the event, reads `CURRENT` and the books up to it, and swaps its immutable snapshot. A lost event is covered by its periodic check of `CURRENT`.

**Operational endpoints.** Every service answers `GET /status` without touching Hazelcast or MongoDB, so a healthcheck never fails because a dependency is down; the Docker healthchecks and `depends_on` use it. The Control adds `GET /control/status` with the books per state and the size of each queue, the observation point of the whole pipeline (Section 4).

# 4. API design and contracts

The full specifications are in the repository, one OpenAPI document per service because every service has a `GET /status`: `api/openapi.yaml` (Search), `api/indexing-openapi.yaml`, `api/ingestion-openapi.yaml` and `api/control-openapi.yaml`. The inventory of every endpoint is `api/endpoints.md`. This section summarises them.

## 4.1 Front-end use cases

A front-end needs three things, and it reaches them only through nginx on port 8080:

- Search books by keywords, optionally by author, language and year, one page at a time.
- Show the details of one book.
- Offer the available filters: the languages and the range of years of the published index.

| Endpoint | Provider | Purpose |
|---|---|---|
| `GET /search` | Search | Keyword search (AND) with filters and pagination |
| `GET /books/{book_id}` | Search | Metadata of one published book |
| `GET /search/filters` | Search | Languages and year range of the published index |

Table 2: Front-end inventory. All are read-only, so they are safe to retry.

## 4.2 Internal and operational interfaces

These are consumed by another service or by an operator, never by the front-end. Only the Control Module is published on the host, on `127.0.0.1`; the others are reached by name inside the Docker network.

| Endpoint | Consumer | Completion |
|---|---|---|
| `POST /ingest/{book_id}` | operators, validation | Done: the book is stored, or was already available |
| `GET /ingest/status/{book_id}` | operators, validation | Done (read-only) |
| `GET /ingest/list` | operators, validation | Done (read-only) |
| `POST /index/update/{book_id}` | operators, validation | Done: indexed, but not visible until published |
| `POST /index/publish` | Control Module | Done: `CURRENT` points to the returned version |
| `POST /index/rebuild` | operators | Accepted: continues in the background |
| `GET /index/status` | Control, operators | Done (read-only) |
| `POST /control/jobs` | operators, benchmarks | Accepted (202): follow it with `/control/status` |
| `GET /control/status` | operators, benchmarks | Done (read-only) |
| `GET /control/tasks/{book_id}` | operators, validation | Done (read-only) |
| `POST /control/retry-failed` | operators | Accepted: the books are back in their queue |
| `GET /status` (every service) | Docker healthcheck, operators | Done. Liveness only: it does not depend on MongoDB or Hazelcast |

Table 3: Internal inventory, and the operational endpoint in the last row.

### 4.3 Parameters, pagination and errors

`GET /search` requires `q`, tokenised with the same `Tokenizer` as Indexing, so a query made only of stopwords has no terms and is rejected. `author` matches part of the name, ignoring case; `language` is an ISO 639-1 code; and `year` has four digits. Filters combine with AND.

Pagination uses `offset` (default 0) and `limit` (default 20, at most 100). `total` counts every match, not only the page, and a valid search without matches answers 200 with an empty list. Results are ordered by ascending `book_id`, behind a `ResultRanking` interface that lets the order change without touching the search. A ranking by relevance would also need term frequencies, which the index does not store.

Every response of Search carries `index_version`, the one version it was computed from. All services share one error format:

```
{"error": {"code": "INVALID_PARAMETER", "message": "limit must be ...", "field": "limit"}}
```

| Status | Code | Meaning |
|---|---|---|
| 400 | `MISSING_PARAMETER`, `INVALID_PARAMETER` | The request is wrong; `field` says where |
| 404 | `NOT_FOUND` | Unknown book, or a route that does not exist |
| 503 | `INDEX_NOT_READY` | Nothing has been published yet |
| 503 | `STORAGE_UNAVAILABLE` | MongoDB does not answer; retry later |
| 503 | `COORDINATION_UNAVAILABLE` | The Control has no connection to Hazelcast |
| 500 | `INTERNAL_ERROR` | Our bug: the details go to the log, not to the client |

Table 4: Main errors. Each specification lists those of its service.

### 4.4 Contracts that are not HTTP

Most of the work between services does not use HTTP:

- **Coordination** (`api/contracts/coordination.md`): the Hazelcast map `book-state`, the queues `ingest-queue` and `index-queue`, the topic `index-events` with the `IndexPublished` event, the lock of the index writer, leases and the transitions of a book.
- **Datalake readiness** (`api/contracts/datalake-readiness.md`): ranges of folders and the `<id>._READY` marker, written last, which says a book is complete.
- **Publication in MongoDB** (`api/contracts/index-publication.md`): `index_books`, `index_meta` (`CURRENT`) and `index_publications`. Switching `CURRENT` changes one document, so a new version appears all at once. The index itself is not copied, because the entries of a written book never change.

### 4.5 Validation evidence

The APIs can be checked without a front-end at three levels, all with their expected results:

- `scripts/validate-api.sh`: every check with one command against the running system, ending with exit code 1 if any fails. On the 15 books of the sample dataset it gives 64 correct, 0 failed and 0 skipped, both with the books loaded directly and with the same books processed by the complete flow (Control, Ingestion, Indexing and publication).
- `StackValidationTest`: the same contract in JUnit, also checking the shape of the JSON.
- `http/*.http`: examples to run by hand, documented in `http/README.md`.

They cover keyword search, each filter alone and combined, valid searches without results, invalid parameters, pagination, the internal endpoints, and the read-only mounts of the datalake. A check only reports *skipped* for an endpoint that answers 501; a route that disappears is a failure. The expected answers were compared with the original texts, so they are not a copy of what the API returns. The load tests refuse to measure if this validation fails.

## 5. Microservice implementation

### 5.1 Ingestion and Control Module

Both are written in Java 17 with Javalin 6.1.3 and Gson, like the rest, and share `common`: the HTTP server with its error format (`ServiceServer`), the configuration from the environment (`Env`), the

range datalake and the coordination library. Ingestion implements `api/ingestion-openapi.yaml` and the Control `api/control-openapi.yaml`; both are checked against their specification by tests (`OpenApiContractTest`) and by `scripts/validate-api.sh` on the running stack.

### 5.1.1 Ingestion Service

| Class | Responsibility |
|---|---|
| `IngestionApi` | `POST /ingest/{id}`, `GET /ingest/status/{id}`, `GET /ingest/list`, `GET /status` |
| `IngestionService` | One ingestion, without HTTP: locate, fetch, split, save. The API and the workers run this same code |
| `GutenbergClient`, `LocalFileSource` | The two sources (`INGESTION_SOURCE`): the download, or `pg<id>.txt` files of a folder |
| `BookSplitter` | Header and body between the `*** START` and `*** END` markers, migrated from Stage 1 |
| `IngestionWorkers`, `IngestionWorker` | The `INGESTION_WORKERS` consumers of `ingest-queue` |
| `Throttle` | One pause for the whole replica after a 429 or a 403 of Gutenberg |
| `BackgroundConnection` (common) | Connects to Hazelcast without blocking the start, and follows the connection |

Table 5: Structure of the Ingestion Service.

**Two ways in, one code path.** `POST /ingest/{id}` is synchronous: when it answers 200 the book is complete in the datalake (`downloaded`, or `already_available` if it was there). It is for administration and for the validation. In normal operation the books arrive through `ingest-queue`, and each worker calls the same `IngestionService`, so both paths store a book in exactly the same way.

**The source.** `GutenbergClient` downloads `https://www.gutenberg.org/cache/epub/<id>/pg<id>.txt` with an identifying User-Agent. Its timeout (30 s) bounds the whole download: the timeout of the JDK client alone covers only the headers, and a body that arrives slowly was measured to last far longer. A 404 is `NOT_FOUND`, a permanent error; a timeout, a network error or a 5xx is a transient one; a 429 or a 403 means that Gutenberg is throttling us. `LocalFileSource` reads the same text from a folder, untouched, so the benchmarks and the synthetic dataset (DR-16) run every step after the fetch exactly as online, without the network.

**Readiness of the datalake** A book is in the datalake only when its `<id>._READY` marker exists together with its header and its body (`api/contracts/datalake-readiness.md`). Each file is written to a temporary file with a unique name and moved into place with an atomic move, and the marker is written last, with the time of the save. So:

- a process that dies halfway leaves files without a marker: the book does not exist for anyone, and the next ingestion writes it again, whole;
- Indexing checks the marker before reading a book (`NOT_READY` otherwise), and the Control's recovery counts only books with a marker;
- a ready book is never written again: `save` is a no-op and the ingestion answers `already_available` without fetching anything.

`locate` computes the path from the id, without listing any folder, so checking one book costs the same with 200 books or with 100,000.

**Task ownership and leases** A worker owns a book from its claim to its completion. The claim polls `ingest-queue` and writes `INGESTING` with the worker's name (`ingestion-<hostname>/w<n>`; inside Docker the hostname is the container id) and `leaseUntil = now + LEASE_SECONDS` (120 s), in one transaction. If the process dies after the poll, the transaction is rolled back and the id goes back to the queue; if it dies after the commit, the book stays `INGESTING` with a lease that nobody renews.

The Control queries `state IN (INGESTING, INDEXING) AND leaseUntil < now` every `LEASE_CHECK_SECONDS` (10 s) and requeues those books with one more attempt (`LEASE_EXPIRED`). The completion is a compare-and-set against the entry the worker claimed: if the Control requeued the book meanwhile, the worker's completion fails, it drops the book, and whatever it wrote is harmless because writing is idempotent. The library can renew a lease (`Coordination.renew`), but Ingestion does not need to: one attempt lasts at most $3 \times 30 + 3$ s with the defaults (the download and two retries, each bounded by the 30 s timeout, plus their pauses of 1 and 2 s), well within the 120 s lease. So a lease only expires when a process really died or hung.

**Retries and errors** Each failure goes back to the contract with a code (§4.3 of the coordination contract):

- **Permanent** (`NOT_FOUND`, `INVALID_BOOK` for a text without markers): `FAILED` at once. Retrying would give the same answer.
- **Transient** (`DOWNLOAD_FAILED`): first `INGESTION_FETCH_RETRIES` (2) retries inside the attempt, after 1 s and 2 s; then the book goes back to the queue with one more attempt, and after `MAX_ATTEMPTS` (3) it is `FAILED`. `POST /control/retry-failed` gives the failed books another round.
- **Throttling** (429 or 403): not retried. The book goes back to the queue as a transient failure, and the whole replica stops claiming for a pause of 30 s that doubles at each new throttling, up to 10 min (a successful download brings it back to 30 s). Otherwise every worker would keep hitting Gutenberg and burn the attempts of every book in the queue in a few seconds; with the pause they keep them.
- **Hazelcast down:** the worker waits and tries again with a growing pause (1 to 30 s); the client reconnects by itself.

**Idempotency** Repeating any step leaves the same result: seeding uses `putIfAbsent`; a book already ready is not fetched; each file is replaced atomically by a complete version, so two ingestions of the same book at the same time (an administrator's `POST /ingest` and a worker) leave one complete copy; and an id that appears twice in the queue is dropped by the claim, because its entry is no longer `PENDING` (counted in `ids_skipped`). This is what makes the retries and the lease recovery safe: a book processed twice costs time, never a wrong result.

**Start and status.** The HTTP server starts first, then the Hazelcast client connects in the background (`BackgroundConnection`, shared with the Control and Indexing), and the workers start once it is connected. `GET /status` answers from the first moment and shows `coordination` (`connecting`, `connected`, `disconnected`, `failed` or `closed`) and the counters of the workers, including `coordination_cost`: the wait in the queue, the claim, the work and the completion of each book (CF-12). A `SIGTERM` (`docker stop`) lets each worker finish its book for up to 10 s before the client and the server close.

### 5.1.2 Control Module

| Class | Responsibility |
|---|---|
| `ControlApi` | `POST /control/jobs`, `GET /control/status`, `GET /control/tasks/{id}`, `POST /control/retry-failed` |
| `ControlTasks` | The lease check and the decision to publish |
| `ControlScheduler` | Runs both tasks on their own threads; a failure of one run is logged and the next run goes on |
| `Recovery`, `StorageEvidence` | Rebuild `book-state` when the cluster lost it (CF-10) |
| `IndexingClient` | `POST /index/publish`, with a 30 s timeout |

Table 6: Structure of the Control Module.

**The API.** `POST /control/jobs` takes 1 to 100,000 ids and answers 202 with how many were new and how many were already known: it only seeds, the work happens asynchronously in the workers. `GET /control/status` counts the books per state and the ids in each queue, and shows the last publication, the last lease check and the last recovery. `GET /control/tasks/{id}` shows the entry of one book: owner, lease, attempts and last error. While the Control is not connected to Hazelcast these routes answer 503 `COORDINATION_UNAVAILABLE` at once instead of waiting for the cluster.

**Publication of completed index updates.** The Control decides *when* to publish; Indexing does the publication. Every second the Control counts the `INDEXED` books, and calls `POST /index/publish` when there are `PUBLISH_BATCH` of them (100) or when `PUBLISH_INTERVAL_S` (30 s) have passed with at least one. A failed call (Indexing down, a timeout, 409 during a rebuild, 503 without MongoDB) is retried after 1, 2, 4… up to 30 s, not every second; the publication is idempotent, so a retry never duplicates a version. A publication that makes nothing new visible (the books were already published but T10 has not marked them yet) does not trigger another one until the interval is over, or until the `INDEXED` books drop below the batch.

**Recovery when Hazelcast loses its state** Hazelcast keeps everything in memory, so if both members stop the state of every book is gone. The Control writes a marker in `coordination-meta` after its first seed or recovery; at start and before every lease check it looks for it. If it is missing, it rebuilds `book-state` from the persistent evidence, from the most advanced state down (contract §7, row R):

1. the books of `index_books` with a version up to `CURRENT` are `PUBLISHED`;
2. the rest of `index_log` are `INDEXED`;
3. the rest with a `_READY` marker are `DOWNLOADED`, and offered again to `index-queue`;
4. the seed file, if any, is seeded again as `PENDING`;
5. the marker is written.

Every source is read before anything is written, a recovered book never goes backwards, and an entry that already exists is only moved forward from `PENDING`. So a recovery interrupted halfway can simply run again. What has no evidence is lost from the cluster: a book that was `PENDING`, or `INGESTING` without its marker. It is sent again with `POST /control/jobs`, which leaves the recovered books as they are. The failure tests of CF-11 restart the whole cluster in the middle of a run and check this end to end (Section 6.1.5).

## 5.2 Indexing

Indexing turns the complete books of the datalake into the inverted index and the metadata in MongoDB, and decides when a new version of the index becomes visible to Search. It is written in Java 17 with Javalin and Gson. Its HTTP API, `api/indexing-openapi.yaml`, has five endpoints: `POST /index/update/{book_id}` indexes one book, `POST /index/publish` and `POST /index/rebuild` publish and rebuild the index, and `GET /index/status` and `GET /status` report its state. Its MongoDB side is specified in `index-publication.md` and its Hazelcast side in `coordination.md`, both in `api/contracts/`. Its main classes are:

| Class | Responsibility |
|---|---|
| `IndexingApi` | Validates the requests and turns results and exceptions into the HTTP answers of the contract |
| `IndexingService` | Reads, parses and tokenizes books, and writes them: one book (`update`) or a batch (`indexBatch`) |
| `IndexingWriter` | The consumer of `index-queue`: holds `index-writer`, claims batches, completes or fails each book |
| `LeaseRenewer` | Renews the leases of the batch being written |
| `Publisher` | Publishes a new version of the index with a conditional switch of `CURRENT` |
| `HazelcastPublicationNotifier` | After a publication, announces `IndexPublished` and marks the books `PUBLISHED`, in a thread of its own |
| `Rebuilder` | `POST /index/rebuild`: rebuilds the whole index into a new collection and switches to it |
| `MongoPublicationRepository` | The publication collections: `index_meta`, `index_books`, `index_publications`, `index_log` |

Table 7: Structure of the Indexing Service. The datalake, the tokenizer, the metadata parser and the MongoDB stores of the index and the metadata are in `common`, shared with the other services.

**One book, one batch.** Both ways in index the same way, in two steps. *Prepare* reads the header and the body of each book from the datalake, parses the metadata (title, author, language, year) and tokenizes the body with the tokenizer of Stage 1, the same one Search uses for queries: ASCII letters in lower case and digits, tokens of at least 2 characters, without stopwords. It runs in parallel on a pool of `INDEXING_THREADS` (4). *Write* runs in one thread and in this order: the metadata of every book of the batch with one bulk write into `books`, the posting lists with one bulk write into the inverted index, and *last* one entry per book in `index_log`. The inverted index sends one `$addToSet` upsert per *distinct* term of the batch, so a common word is written once per batch, not once per book. A book that cannot be prepared is left out and reported, and never stops the others. A write that fails fails the whole batch.

**Why `index_log` goes last.** It is the evidence that a book is complete in the index: the publication takes its books from it, the book becomes `INDEXED` in Hazelcast only after it, and the recovery of the Control (Section 5.1.2) reads it. A process that dies in the middle of a batch leaves no entry, so the book is indexed again and never published half written.

### 5.2.1 Readiness, task ownership and retries

**Readiness.** Indexing reads a book only if its `_READY` marker is in the datalake (Section 5.1.1). Otherwise `POST /index/update` answers 404 and the queue writer fails the book with `NOT_READY`, which puts it back in the queue.

**Task ownership.** One thread per replica runs `IndexingWriter`. It first takes the Hazelcast lock `index-writer`: only its holder consumes `index-queue`, and another replica waits as a standby and takes over when the holder stops or dies. The holder claims a book, waiting for one, and then the books already waiting, without waiting, up to `INDEX_WRITE_BATCH` (100). With few books a batch has one book and nothing waits for more; with a backlog the batches fill up by themselves. After the write, it completes each indexed book (`INDEXED`). Each claim has a lease of `LEASE_SECONDS` (120 s). A batch on a large index can take longer than that (about 33 s per batch at 10,000 books, Section 7.2), so `LeaseRenewer` renews the leases of the batch every third of the lease while it is written. Without it, the Control would requeue books that are still being written.

**Retries.** Each failure goes back to Hazelcast with a code of the coordination contract:

| Failure | Code | What happens |
|---|---|---|
| Book not complete in the datalake | `NOT_READY` | Requeued, one attempt more |
| Book without indexable terms | `EMPTY_BOOK` | `FAILED` at once; nothing is written |
| MongoDB does not answer | `STORAGE_UNAVAILABLE` | The whole batch is requeued *without* counting an attempt, and the writer waits 1, 2, 4… up to 30 s before its next claim |
| Anything else | `INTERNAL_ERROR` | Requeued, one attempt more |

Table 8: Failures of the queue writer.

If completing or failing a book finds that its task changed, the writer had lost it (the Control requeued it after its lease expired), and the result is dropped. When the client loses the cluster, the writer stops claiming at once, and takes the lock again before writing anything when the cluster is back. In real failover runs, a standby replica took over 2.8 s after a `SIGTERM` of the holder and 4.0 s after a `kill -9`, and the 17 books the dead holder had claimed came back through their leases.

### 5.2.2 Idempotency

Indexing a book twice leaves the index exactly as indexing it once, so every retry is safe:

- the posting lists use `$addToSet`, so an id is never added twice, and the unique index on `term` keeps one document per term;
- `books` and `index_log` are upserts by `_id`;
- two replicas, or two threads, writing the same new term at once meet on the unique index. MongoDB retries such an upsert by itself because its filter is an equality on the unique key, so neither fails.

A test with two `IndexingService` of 4 threads each, indexing the same books at once against a real MongoDB, checks that the index ends as after a single run. `docs/robustness.md` lists each failure, what must happen and the test that proves it, and the mutants those tests catch.

### 5.2.3 Publication of completed index updates

Search must never show a book half written, and must see all the books of a version at once. A version is a document `CURRENT` in `index_meta`, and a book is visible when `index_books` gives it a version $\leq$ `CURRENT`. `Publisher` publishes on top of `CURRENT` = V in four steps:

1. the books to publish are those of `index_log` that are not in `index_books`: complete by construction;
2. it records them in `index_books` with version V+1, with `$setOnInsert`: a book keeps forever the first version in which it became visible. Search still shows V;
3. it switches `CURRENT` to V+1 *only if it is still V*, a conditional update. This is the instant the books become visible, all at once;
4. it writes the entry of V+1 in `index_publications`, the history.

A process that dies between steps 2 and 3 leaves books above `CURRENT`, invisible, and the next publication completes them. If the conditional switch fails, another replica published first, and the publication is repeated on top of the new `CURRENT`, up to 3 times. The Control decides *when* to publish (Section 5.1); Indexing does it.

After answering, the publication is announced from a thread of its own: the `IndexPublished` event on `index-events`, one per version and in order while connected, and the transition of the published books from `INDEXED` to `PUBLISHED` in Hazelcast. A slow Hazelcast therefore never holds the publication nor its HTTP request. Without the cluster the last version is kept and announced on reconnection; a lost event does no harm, because Search also checks `CURRENT` every 5 s.

### 5.2.4 Full rebuild of the index

`POST /index/rebuild` indexes every complete book of the datalake again into a *new* collection, `inverted_index_rNN`, while Search keeps reading the current one. It is needed when the tokenizer changes, for instance. Any replica can start it:

1. it puts a marker in Hazelcast (`putIfAbsent` with a TTL of 30 s), so only one rebuild runs. While the marker exists no writer claims from the queue, and update and publish answer 409 in every replica. A thread of the rebuild renews the TTL every 10 s, also in the middle of a batch; if the replica dies, the marker expires and indexing goes on;
2. it takes `index-writer`, which the queue writer releases at the end of its batch, and indexes every book into `rNN` in batches;
3. a safety pass indexes into `rNN` every book published meanwhile in the old collection (a replica without Hazelcast does not see the marker);
4. it switches with one more publication, V+1 on `rNN` with every book: Search sees the new collection, and every book, at once;
5. the old collection is dropped after `INDEX_REBUILD_DROP_SECONDS` (60 s), well above the 5 s in which a Search replica still serves its old snapshot.

If anything fails before the switch, `rNN` is dropped and `CURRENT` is untouched.

**Availability.** `GET /status` never touches MongoDB, so it answers from the first second. The connection to Hazelcast is made in the background after the HTTP server starts; until then, and whenever the cluster is lost, `POST /index/update` and `POST /index/publish` keep working, because indexing a book does not need the cluster. If MongoDB does not answer, they answer 503 `STORAGE_UNAVAILABLE`, and retrying is safe.

## 5.3 Search and deployment

### 5.3.1 Search Service

Search is written in Java 17 with Javalin and Gson, and implements `api/openapi.yaml`. It never writes and mounts no volume: everything it needs is in MongoDB, which makes it easy to move to another machine in Stage 3. Its main classes are:

| Class | Responsibility |
|---|---|
| `SearchApi` | Validates the parameters and writes the HTTP answers |
| `SearchEngine` | Runs one query on one snapshot |
| `IndexSnapshotLoader` | Builds a snapshot of the published version from MongoDB |
| `IndexSnapshot` | Immutable: version, published ids and their metadata |
| `SnapshotHolder` | Keeps the current snapshot in an `AtomicReference` |
| `SnapshotRefresher` | Checks `CURRENT` periodically and on demand, on one thread |
| `PublicationEvents` | Listens to `IndexPublished` in the background |
| `MongoPostingSource` | Reads the posting lists of the terms with one query |

Table 9: Structure of the Search Service.

**One query.** `SearchEngine` reads the posting lists of the terms. It walks the shortest list and keeps an id only if every other list has it, using binary search, so the cost depends on the rarest term. Then it drops the ids that the snapshot does not publish, applies the filters on the metadata in memory, orders all the matches and cuts the requested page.

**Consistency.** Indexing may already have written books that are not published, so Search must not show everything it finds. Each request takes the snapshot once when it starts and works only with that one, so it cannot mix versions, and it reports that version as `index_version`. The loader reads `CURRENT` first and then the books up to that version. A book keeps the version in which it first became visible, so whatever moment the loader runs, it sees all of a version and never part of the next one. Replacing the snapshot is one atomic write: no request waits for it, and no thread can see one half built.

**Reloading.** A new version reaches Search in two ways. The `IndexPublished` event makes it react at once, but it is only a wake-up: its content is not used, and whoever receives it reads `CURRENT` from MongoDB. The check of `CURRENT` every `SEARCH_POLL_SECONDS` (5 s) is the backup for lost events. Both run the reload on the same thread, so a slow reload never blocks the Hazelcast thread and two reloads never overlap. If a reload fails, the replica keeps serving the snapshot it has.

A failure found in testing shows why both are needed. The ringbuffer of a new Hazelcast cluster starts again at sequence 0, while the subscription keeps waiting at the sequence it had reached. After restarting both members, Search received no event until as many versions had been published, and only the periodic check kept it up to date. Each reconnection now subscribes again and checks `CURRENT` once; a test restarts the cluster repeatedly and publishes one event at a time, as Indexing does.

**Availability.** Search starts even if MongoDB or Hazelcast are down, or nothing has been published. `/status` answers from the first moment, because connecting to Hazelcast happens in the background after the HTTP server starts. Searches answer 503 until a version is loaded. If Hazelcast is down, Search keeps working with the periodic check; if MongoDB is down, searches answer 503 `STORAGE_UNAVAILABLE` and recover by themselves.

**Concurrency.** A request runs on a thread of the Jetty pool, whose maximum is explicit in `SEARCH_HTTP_THREADS`: 250, the default value of Javalin 6.1.3, with a minimum of 8 and a 60 s

idle timeout. These threads share only immutable data, so they need no lock. One `MongoClient` per process has a pool of `SEARCH_MONGO_POOL_SIZE` connections (50), and a request that waits more than 5 s for one answers 503 instead of keeping its thread busy for the 2 minutes of the driver's default.

**Browsers.** `CORS_ORIGINS` lists the origins allowed to call the API from a web page; an invalid value stops the start.

### 5.3.2 Deployment

`docker compose up` starts everything on one host from `docker-compose.yml` and `.env`, which holds every default. All Java services use one multi-stage template, `docker/service.Dockerfile`, with the Maven module as an argument.

| Service | Replicas | Published on the host | Datalake | Limits |
|---|---|---|---|---|
| `nginx` | 1 | 8080 | none | 0.5 CPU, 128 MiB |
| `search` | 1–4 | none | none | 2 CPU, 1 GiB each |
| `indexing` | 1–2 | none | read-only | 2 CPU, 1 GiB each |
| `ingestion` | 1–4 | none | read-write | 1 CPU, 512 MiB each |
| `control` | 1 | 7000, local only | read-only | 0.5 CPU, 384 MiB |
| `hz-1`, `hz-2` | 2 | 5701, 5702, local only | none | 1 CPU, 768 MiB each |
| `mongo` | 1 | 27017, local only | none | 2 CPU, 1 GiB |

Table 10: Containers of the deployment. One replica of each adds up to about 5.7 GB of memory limits.

- **Replicas.** nginx (`scripts/nginx.conf`) does round-robin over the replicas of Search, which it finds with the DNS of Docker and looks up again while running, so three replicas only need `docker compose up -d --scale search=3`.
- **Replicas of Indexing.** Only the replica that holds the Hazelcast lock `index-writer` writes; the others wait and take over if it stops or dies. Two replicas therefore give availability, not more indexing.
- **Failures.** The public API is read-only, so nginx can safely repeat a request on another replica if one dies. It does not do so on a 503: a replica that answers is alive, and the others would answer the same. Killing one of 3 replicas during a test lost none of 2,000 requests.
- **Permissions.** Only Ingestion can write in the datalake, and Docker enforces it. Hazelcast has no authentication, so only nginx listens on all the interfaces.
- **Fixed resources.** Every container has limits, and the heap is a fixed share of its memory, so the benchmarks can be repeated and report what they used.
- **Monitoring.** `scripts/monitor.sh` records the CPU and memory of each container and the load of the host as CSV. A sample of the containers takes about 2 s, because of `docker stats`.

**Replicas and threads are different things.** A thread shares its process with the others, including the snapshot. A replica is another process with its own memory, its own snapshot and its own connection pool, reached through nginx. Both share the CPU, memory and disk of the host, and all the replicas share one MongoDB.

# 6. Design decisions

## 6.1 Coordination

### 6.1.1 Why Hazelcast

The services need four things from their coordination: queues of work that several replicas can share, a state per book that several processes change at the same time, one lock (the single index writer) and one event (a new version). Table 11 compares the options we considered.

| Alternative | Why we did not choose it |
|---|---|
| Control files of Stage 1 | Plain text files do not survive several processes writing at the same time |
| Orchestrator with SQLite | It works, but the orchestrator calls each replica and must know its address; with shared queues, adding a replica changes nothing in the Control |
| Broker (RabbitMQ, Kafka) | Acknowledgements and persistence, but one more technology, and no shared state, locks or compare-and-set: the state of each book would need yet another store |
| MongoDB for coordination too | It could keep the state of each book, but it has no blocking queues, topics or distributed locks; polling a collection for work would load the database that Search reads |
| Shared memory (`mmap`) | Only exists on one machine: the assignment would force a redesign for Stage 3 |
| **Hazelcast** | Queues, a map with compare-and-set and queries, transactions, locks and a reliable topic in one dependency. It is used the same way on one machine and on a cluster: for Stage 3 the configuration changes, not the code |

Table 11: Coordination alternatives.

Hazelcast has three limits, which shaped the rest of the design:

1. **It is only in memory.** If both members stop, every state is lost. Hence the principle that a state is written only after its evidence is on disk, and the recovery of the Control (Section 5.1.2).
2. **Its queues have no acknowledgements.** An id taken by a worker that dies is gone from the queue. Hence the leases in `book-state`, reviewed by the Control (Section 5.1.1).
3. **Its strongly consistent primitives are not free.** The CP Subsystem, and with it `FencedLock`, is an Enterprise feature in 5.7: `getCPSubsystem().getLock()` throws `UnsupportedOperationException` (measured before writing the contract). The index writer uses `IMap.lock` instead, which has no fencing token.

### 6.1.2 How the shared state is protected

**One library.** Every access to Hazelcast is a method of `Coordination` in `common`, one per transition of the contract. No service names a structure or builds a value by hand, so the rules below hold everywhere by construction, and they are tested once against embedded members.

**Compare-and-set, not locks.** A transition reads the entry, builds the next value and writes it with `replace(id, expected, next)`, which the member applies only if the stored bytes are still the expected ones. Two workers that race for a book cannot both win, and nobody holds a lock while it downloads. We did not use an `EntryProcessor`, which would run our code on the members: the members run the official image without our classes, and the values are Compact records that the members can store, compare and query field by field without them.

**Transactions where a queue is involved.** Claiming a book (poll and `INGESTING`) and completing an ingestion (`DOWNLOADED` and the offer to `index-queue`) each touch a map and a queue. They run in one `TWO_PHASE` transaction, which survives the death of a member during the commit. Without it, a crash between the two operations would leave a book `DOWNLOADED` that no queue holds, or an id taken from the queue that no state records.

**Duplicates are harmless.** The queues are work to do, `book-state` is the truth. An id can be twice in a queue (a requeue racing with a late completion, a seed repeated by hand): whoever takes it checks its state in the claim and drops it if it is not waiting. Together with idempotent work (atomic files, upserts, `$addToSet`, `putIfAbsent`), this is why at-least-once delivery is enough and exactly-once is never needed.

**Fencing where it matters.** `IMap.lock` has no fencing token: a writer that froze and comes back late is not stopped by Hazelcast. For the index this costs at most repeated work, because its writes are idempotent and its claims fail once the Control has requeued its books. A publication is not idempotent (a late writer could move `CURRENT` back), so Indexing uses the version itself as the token: `CURRENT` only moves from V to V+1 with a conditional update, and a publication whose update matches nothing is aborted.

### 6.1.3 Single machine, replicas, threads and ports

**Two Hazelcast members on one host.** One member would be enough on one machine; two show the behaviour of the cluster that Stage 3 needs: partitions spread over members, one backup of every entry, a member that dies without losing anything, and clients that move to the other member. On one host they share its failures, so this is evidence of the mechanism, not real high availability.

**Ingestion: replicas and workers.** Ingestion scales by adding processes (`--scale ingestion=N`) or threads (`INGESTION_WORKERS`). Both pull from the same queue, so the work is shared without any configuration, and correctness does not depend on the counts. Downloading is mostly waiting on the network, which favours several workers per process (4 by default); the limit is Gutenberg, which throttles a client that downloads too fast. With the local source the work is CPU and disk, and each container has one CPU, so beyond a few workers only more replicas help.

**One Control.** It is not in the path of any book, so one is enough and its absence only delays leases and publications. A second one would be safe, because every transition is a compare-and-set and the publication is idempotent, but it would double the lease queries and the calls to Indexing for nothing. Stage 3 needs a leader election instead (Section 8).

**Ports.** Only nginx is published to the network. The Control, the Hazelcast members and MongoDB have no authentication, so they are published on `127.0.0.1` only, for the administrator, the tests and the services run from an IDE. Ingestion and Indexing are reached only inside the Docker network. The clients use `SINGLE_MEMBER` routing: the members announce their Docker-internal addresses, which a process on the host cannot reach, so a client talks only to the addresses it was given and moves to the next one if its member dies.

### 6.1.4 Where replication helps and where it contends

- **`book-state` is partitioned** by book id over the 271 partitions of the cluster, so claims of different books touch different partitions and do not contend.
- **Each queue lives in one partition**, on one member: every claim of every worker goes through it. This serializes the claims, and it is the first point of contention of the coordination when the work per book is small (the local source). With Gutenberg the download is far longer than a claim, so it should not show there.

- **Each book costs two transactions** (claim and completion), each with the round trips of a two-phase commit. That is the coordination cost per book, measured on its own with JMH and inside the running system with the metrics of CF-12.
- **The datalake is one disk.** More Ingestion replicas write to the same file system; each book is three small files and three atomic moves in the same range folder.
- **Indexing has a single writer by design.** More Indexing replicas would only be standbys; its parallelism is in the threads that parse, inside one process.

**Measured.** With JMH (Section 7.1), claiming and completing a book costs 1.6 ms with one worker, 3.4 times the local ingestion of the book, and 0.36 ms per book with 8 workers, because concurrent workers overlap their round trips. So with small books the coordination, not the work, sets the rate of a replica; with the download from Project Gutenberg it is the other way round. The contention of several replicas on the single partition of each queue was not measured: the integration experiments of Ingestion were not run.

### 6.1.5 Failure tests

The scenarios of `scripts/chaos/` (CF-11) cause each failure in the middle of a real run on Docker Compose and check the end state against the contract. Each one leaves a `report.md` with what was expected, what happened and the evidence (the states, a timeline of the books and the logs of every service).

| Failure | Expected | Result |
|---|---|---|
| SIGKILL of one of two Ingestion replicas while it ingests | Its books are requeued when their lease expires and the other replica finishes them; no book `FAILED` or duplicated | **PASS.** Killed while holding a book; its lease expired and the Control requeued it; 2,000 of 2,000 `DOWNLOADED` |
| SIGKILL of one Hazelcast member | Nothing lost (backup), the clients carry on, no recovery; the member rejoins | **PASS.** `hz-2` carried on alone with 1,848 books to go; no rebuild; `hz-1` rejoined |
| SIGKILL of the Control while Indexing indexes | The workers keep going, nothing is published until it is back; it rebuilds nothing | **PASS.** Indexing went from 100 to 500 books, `index_books` stayed at 100; no rebuild |
| Both members killed and started empty | The Control rebuilds `book-state` from the evidence; the books without evidence are sent again; none with `_READY` is ingested again; all end `PUBLISHED` | **PASS.** Rebuilt in 1.9 s: 3 `PUBLISHED`, 100 `INDEXED`, 336 `DOWNLOADED`; the 1,561 without evidence sent again, none ingested twice |

Table 12: Documented failure tests. Each run checks that every book reaches its final state exactly once, that none is `FAILED`, and that every book is complete in the datalake and, with Indexing, in `index_log` and `index_books`.

**How they ran.** On the reference machine, on 8 November, with 2,000 synthetic books per scenario, leases of 15 s and the commit recorded in each `report.md` (`scripts/chaos/results/`). With the local source a book takes milliseconds, so the scenarios that kill during the ingestion

run one worker per replica and check that books were still in flight right after the failure. To be sure that the killed Ingestion replica held a book, it is frozen first (`docker pause`) and killed only while the Control shows its book `INGESTING`. A first run gave three FAILs with every check of the end state passing: two came from errors in the checks themselves, and in the third the killed replica held no book, so no lease was tested. The script was fixed and the four scenarios run again; the table gives that second run.

## 6.2 Storage and concurrency

### 6.2.1 Storage selection

Stage 1 compared several structures for each store and chose, for 200 books, a datalake with one folder per book and a monolithic JSON inverted index. Stage 2 must index tens of thousands of books with several processes sharing the data, so the group moved to structures that Stage 1 had already implemented and tested:

| Store | Choice | Why |
|---|---|---|
| Datalake | *range*: one folder per 1,000 ids, such as `01000-01999/` | With one folder per book the root holds one entry per book, hundreds of thousands. With *range* each folder has at most about 3,000 files (header, body and `_READY` of each book), and the folder of a book is computed from its id, never searched. Files are the cheapest way to keep raw text, and the only store on disk besides MongoDB. |
| Inverted index | MongoDB: one document `{term, postings}` per term, unique index on `term` | The monolithic JSON is held whole in memory and rewritten whole at each flush; Stage 1 itself noted that it had to change beyond about 2,000 books. In MongoDB a flush sends only the terms that changed, a query reads only the documents of its terms, and many processes can read and write it at once. |
| Metadata | MongoDB: collection `books`, one document per book | By volume SQLite was enough (about 30 MB for 100,000 books). The reason is Stage 3: a SQLite file cannot be shared between machines, and MongoDB was already deployed. Search reads the metadata by `_id` and filters in memory. |
| Publication | MongoDB: four collections | `index_meta`, `index_books`, `index_publications` and `index_log`. The version of the index must change for Search in one atomic step, and a conditional update of one document (`CURRENT`) gives it without any lock (Section 5.2). |

Table 13: Storage of each kind of data. The datalake is the only persistent truth on disk; everything else is in one MongoDB database.

**What the measures say about this choice** (Section 7.2). With 30,000 books the index has 651,565 terms and 94.7 million ids, and takes 1.11 GB on disk. Its longest posting list, the word *said*, holds 29,898 ids in 318 KB: far from the 16 MB limit of a MongoDB document, and reading it takes 2.8 ms. One document per term is therefore enough for queries at this scale. The cost is on the write side. `$addToSet` compares each new id with the whole list to keep it free of duplicates, so adding 100 ids costs 6 ms on a list of 1,000 ids and 233 ms on one of 100,000. Since the frequent terms hold almost one id per book, every batch pays for the size of the index, and the indexing rate falls from about 17 to 1 book per second between an empty index and 30,000 books.

**What would fix it, and why it was not done.** `$push` is 14 to 20 times faster on long lists, but it accepts duplicates, and idempotency is what makes every retry of the pipeline safe (Section 5.2). Avoiding duplicates without `$addToSet` would need another mechanism, such as writing a batch only once per book with a check in `index_log`. The other option is to split each posting list into

buckets of bounded size, which keeps `$addToSet` cheap but changes the contract of the index and the reads of Search. Both are proposed for Stage 3, not measured.

### 6.2.2 Concurrency model of Indexing

One Indexing process runs these threads:

| Threads | How many | What they do |
|---|---|---|
| Jetty pool | up to 250 (`INDEXING_HTTP_THREADS`) | One per HTTP request |
| Parsers | 4 (`INDEXING_THREADS`) | Read, parse and tokenize the books of a batch, in parallel |
| Queue writer | 1 | Holds `index-writer`, claims, writes, completes |
| Lease renewer | 1 | Renews the leases of the batch being written |
| Notifier | 1 | Announces each publication and marks its books `PUBLISHED` |
| Rebuild | 1, and 1 for its marker | Only while a rebuild runs |

Table 14: Threads of one Indexing process. The Hazelcast client and the MongoDB driver add their own.

**Many parsers, one writer.** Preparing a book is pure CPU and independent of the others, so it runs in parallel. Writing is not: the pending changes of the inverted index are one map per process, and the order of the writes (metadata, posting lists, `index_log`) is what makes a book complete. So writes go one at a time, whether they come from the queue writer or from `POST /index/update`. Between replicas, the `index-writer` lock leaves a single writer in the whole system; more replicas are standbys for failover, not more throughput. A second writer would update the same documents as the first, because the frequent terms are in almost every book: the two would contend on the same posting lists in MongoDB and add no capacity where it is missing.

**Protection of the shared state.**

| Shared state | Shared by | Protection |
|---|---|---|
| Pending posting lists | Threads of one process | A lock around add and flush |
| Writing the index | Replicas of Indexing | Hazelcast lock `index-writer`, held by one thread |
| Leases of the claimed books | Writer and Control | Conditional replace of the task in `book-state` |
| `CURRENT` | Replicas that publish | Conditional update: only from V to V+1 |
| Version of a published book | Publications | `$setOnInsert`: it never changes |
| Posting lists and metadata | Replicas, retries | Idempotent writes: `$addToSet`, upserts by `_id` |
| Rebuild marker | Replicas | `putIfAbsent` with a TTL and a token; renewed and removed only by its owner |

Table 15: How Indexing protects each piece of shared state.

### 6.2.3 Where threads and replicas help, and where they contend

- **More parser threads help the CPU part only.** Alone, preparing a batch goes from 233 to 487 books per second between 1 and 8 threads, and stops improving at 4. Inside

the deployment the whole path into MongoDB stays at 15 books per second whatever the threads: CPU is not what limits indexing.

- **More Indexing replicas do not add throughput** by design: they are standbys. What they add is failover, measured at 2.8 to 4.0 s.
- **MongoDB is the shared resource.** Indexing writes the index that Search reads. With the pipeline running, Indexing processed 2.42 books per second alone and 0.91 while Search was under load, and with 128 clients the p95 of Search rose from 109 to 160 ms. On one host they also share its CPUs.
- **What would help**, for Stage 3: a cheaper write than `$addToSet` on long lists, a larger MongoDB cache (the index exceeds its 0.25 GB from about 10,000 books), and MongoDB on a machine of its own, away from the other services.

# 7. Benchmarks and results

## 7.1 Ingestion

Ingestion was measured with JMH on the host: the work of a worker on one book, without the network of Project Gutenberg, and apart from it the cost of coordinating that book through Hazelcast. The scripts, the instructions to repeat the measure and the raw results are in `benchmarks/ingestion/`.

### 7.1.1 Microbenchmarks with JMH

The benchmarks live in the test code of `ingestion-service`, package `...ingestion.benchmark`, and run from the benchmarks jar on the reference machine (Section 7.4), with the same settings as Indexing and Search: 2 forks with a 4 GB heap, 5 warm-up and 10 measured iterations of 1 s each (20 samples), average time per operation with its 99.9 % error. The books are the 200 synthetic books of the dataset, the same bytes as the Indexing benchmarks, read from the page cache. The benchmarks with threads take a round of 100 books, shared by 1, 2, 4 or 8 threads as the workers of a replica share `ingest-queue`; every round writes into an empty datalake, and after it, unmeasured, the benchmark checks that the 100 books are complete (or `DOWNLOADED` and in `index-queue`), so a wrong round fails instead of giving a number.

| Benchmark | Time ($\mu$s) | Derived |
|---|---|---|
| Split a short book (8.0 KB raw) | 11.5 ± 1.0 | 695 MB/s |
| Split the median book (51.7 KB) | 79.3 ± 5.8 | 651 MB/s |
| Split a long book (287.8 KB) | 454 ± 15 | 634 MB/s |
| Save a book in the range datalake, per book, 1 thread | 235 ± 5 | 551 MB/s |
| 2 threads | 142 ± 4 | ×1.66 |
| 4 threads | 112 ± 4 | ×2.09 |
| 8 threads | 121 ± 2 | ×1.93 |
| Ingest a book from the local source, per book, 1 worker | 473 ± 8 | 2,113 books/s |
| 2 workers | 270 ± 4 | ×1.75 |
| 4 workers | 175 ± 4 | ×2.70 |
| 8 workers | 155 ± 2 | ×3.06 |
| Claim and complete a book in Hazelcast, per book, 1 worker | 1,631 ± 108 | 613 books/s |
| 2 workers | 1,052 ± 66 | ×1.55 |
| 4 workers | 581 ± 19 | ×2.81 |
| 8 workers | 364 ± 15 | ×4.48 |

Table 16: JMH of Ingestion. *Ingest* is `IngestionService.ingest`: read the raw file, split it, check that the book is not in the datalake and save it (header, body and `_READY`, each through a temporary file and an atomic move). *Claim and complete* are T2 and T3 of the contract, two transactions each, on a cluster of two members started in the same JVM, the threads sharing one client as the workers of a replica do. The factors are the speed-up over 1 thread; the books of a round average 129 KB written.

**Where the time of a book goes.** With one worker, ingesting a book from the local source takes 473 $\mu$s: about 200 $\mu$s to split it (the splitter runs at 630 to 700 MB/s, linear in the size of the book), 235 $\mu$s to write its three files, and the remaining 35 $\mu$s to read it from the page cache and check that it is not stored yet. Nothing is forced to disk (the datalake does not `fsync`, Section 5.1.1), so the write measures the file system up to the page cache.

**Threads.** The ingestion speeds up 2.7 times with 4 workers and 3.1 times with 8, on 6 physical cores. The write stops at 4 threads and is slower with 8: the 100 books of a round, with consecutive ids, go to the same range folder, as they do in the service, and every atomic move changes that one directory. The probable cause is that the file system updates one directory one change at a time; it was not measured apart. The splitting, pure CPU, keeps scaling, which is why the whole ingestion still gains a little at 8.

**Coordination.** Claiming and completing a book costs 1.6 ms with one worker: 3.4 times the ingestion of the book itself. Each one is a transaction with a two-phase commit, so the time is round trips rather than CPU, and concurrent workers overlap them: with 8 workers the cost per book falls to 0.36 ms (×4.5), better than any other part. With the default of 4 workers, coordination takes 581 $\mu$s per book against 175 $\mu$s of work: three quarters of the time of a book with the local source. In this benchmark the members are in the same JVM and talk over loopback, so it is a lower bound of the network cost and an upper bound of the CPU they compete for; the queue is never empty, so `claim` contains no wait for an id.

**What it means for the pipeline.** With the local source, one replica with 4 workers is bounded by Hazelcast at about 1,700 books per second in these conditions. That is two orders of magnitude above Indexing, which does 15 books/s on 2,000 books (Section 7.2), so Ingestion is never the bottleneck of the pipeline. With Project Gutenberg as the source the download is much longer than both parts and the throttling of the site sets the pace; that is why the measures use the local

source.

**Not measured.** The integration experiments of Ingestion with the real services (1, 2 and 4 replicas, the 10,000 books and the coordination cost inside Docker) were prepared (`run-integration.sh`, in the branch `test/CF-13-integration`) but not run: the group left them as not measured. Without them there is no measure of the contention of several replicas on the single partition of each queue (Section 6.1), nor of the overhead of Docker on the coordination; the services expose the needed metrics in `GET /status` (`coordination_cost`, CF-12) for whoever runs them.

## 7.2 Indexing

Indexing was measured at two levels: its CPU work alone with JMH on the host, and the whole path from `index-queue` into MongoDB with the real services in Docker Compose. The scripts, the instructions to repeat every measure and the raw results are in `benchmarks/indexing/`; the figures come from `benchmarks/plot.py`.

### 7.2.1 Microbenchmarks with JMH

The benchmarks live in the test code of `indexing-service`, package `...indexing.benchmark`, and run from the benchmarks jar on the reference machine (Section 7.4): 2 forks with a 4 GB heap, 5 warm-up and 10 measured iterations of 1 s each (20 samples), average time per operation with its 99.9 % error. The books are three synthetic bodies of the dataset (short, median and long), read from the page cache, so the benchmarks measure CPU, not disk.

| Benchmark | Time ($\mu$s) | Derived |
|---|---|---|
| Tokenize a short body (6.8 KB) | 114 ± 8 | 60 MB/s |
| Tokenize the median body (49.6 KB) | 844 ± 66 | 59 MB/s |
| Tokenize a long body (281.5 KB) | 5,137 ± 310 | 55 MB/s |
| Parse a header | 21.2 ± 0.7 | |
| Add a book to the pending posting lists, in memory | 1,320 ± 34 | |
| The same, pending map of the MongoDB index (nothing sent) | 1,343 ± 40 | |
| Prepare a batch of 100 books, per book, 1 thread | 4,297 ± 170 | 233 books/s |
| 2 threads | 3,130 ± 126 | ×1.37 |
| 4 threads | 2,119 ± 87 | ×2.03 |
| 8 threads | 2,052 ± 39 | ×2.09 |

Table 17: JMH of the CPU part of indexing. A batch is read, parsed and tokenized on the parser threads; adding to the posting lists stays in one thread.

The threads stop helping at 4: reading, parsing and tokenizing run in parallel, but adding to the posting lists is serial and takes about 1.3 ms of the 2.1 ms per book. Even with 1 thread, the CPU work of a book (4.3 ms) is a small part of the 66 to 500 ms that a book takes end to end, as the next section shows.

### 7.2.2 Integration experiments

**Method.** `run-experiments.sh` drives the real services of Docker Compose with the limits of `.env`: Indexing with 2 CPUs and 1 GB, MongoDB with 2 CPUs, 1 GB and a WiredTiger cache of 0.25 GB. Each run starts clean: the database is deleted and both Hazelcast members are restarted. With Indexing stopped, Ingestion puts the books of the run in `index-queue`. Then Indexing

starts at time 0, so its start-up counts, and the driver counts the `INDEXED` books every second. `scripts/monitor.sh` samples the CPU and memory of every container every 2 s. The Control is not started, because it would publish in the middle of a measure. The throughput is measured between 10 and 90 % of the books, without the start-up and the last batch. Batches of 100 books; one factor changes at a time; the books are the synthetic ones of the dataset, with real word frequencies. The whole run took 6.5 hours, and no book ended `FAILED`.

**Threads.** 2,000 books, 1 replica, 3 repetitions each:

| INDEXING_THREADS | Books/s | Duration (s) | CPU Indexing | CPU MongoDB |
|---|---|---|---|---|
| 1 | 14.87 ± 0.21 | 137.3 ± 2.5 | 14 % | 101 % |
| 2 | 15.05 ± 0.15 | 135.7 ± 1.1 | 14 % | 102 % |
| 4 | 15.14 ± 0.17 | 135.1 ± 1.2 | 14 % | 104 % |
| 8 | 15.19 ± 0.24 | 134.7 ± 1.7 | 18 % | 101 % |

Table 18: Indexing against its parser threads. CPU is the mean in % of one core during the run. Peak memory: Indexing 476 to 477 MiB, MongoDB 538 to 582 MiB. Mean ± standard deviation of 3 runs.

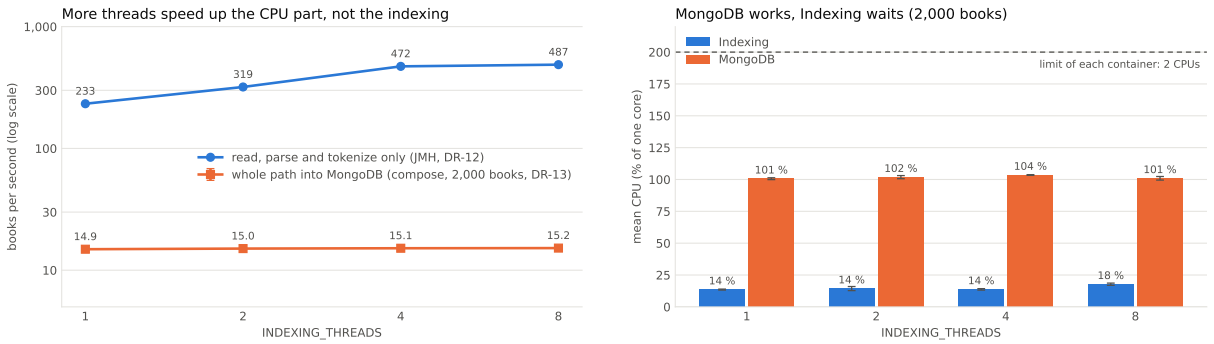



Figure 4: Left: books per second against the parser threads, for the CPU part alone (JMH) and for the whole path into MongoDB. Right: mean CPU of Indexing and MongoDB in those runs.

**Scale.** 4 threads, 1 replica; 30,000 books ran once because one run takes more than 4 hours:

| Books | Runs | Books/s | Duration | Terms | Index on disk | Longest list |
|---|---|---|---|---|---|---|
| 200 | 3 | 24.90 ± 0.02 | 11 s | 58,244 | 5.8 MB | 200 ids |
| 10,000 | 3 | 5.54 ± 0.01 | 30 min | 394,502 | 465 MB | 9,965 ids, 99 KB |
| 30,000 | 1 | 2.00 | 4 h 10 min | 651,565 | 1.11 GB | 29,898 ids, 318 KB |

Table 19: Indexing against the number of books. Peak memory: Indexing 404 to 518 MiB, MongoDB 461 to 802 MiB of its 1 GB, Hazelcast 326 to 390 MiB.

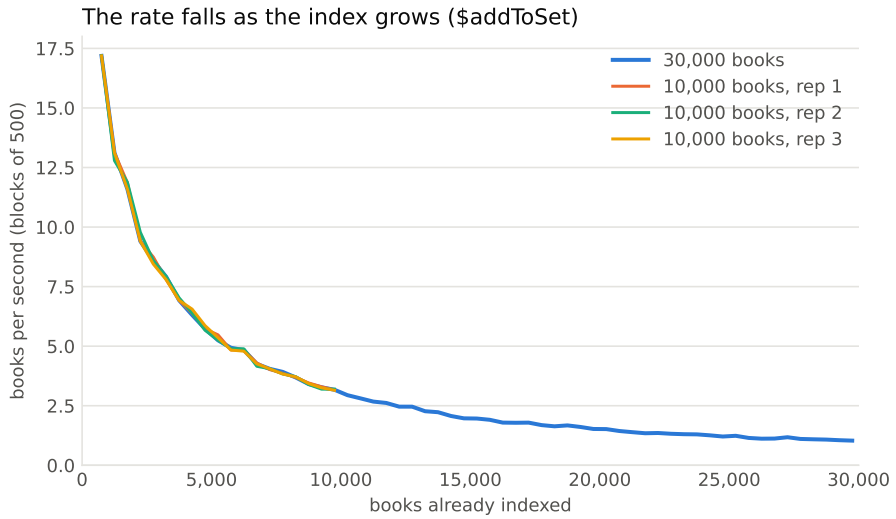



Figure 5: Books per second against the books already in the index, in blocks of 500 books (5 batches). The three runs of 10,000 books and the run of 30,000 lie on one curve.

**Probe of the posting lists.** To isolate the cost of one update, the driver adds 100 ids to a single posting list of a given length, 20 times, with `$addToSet` and with `$push`:

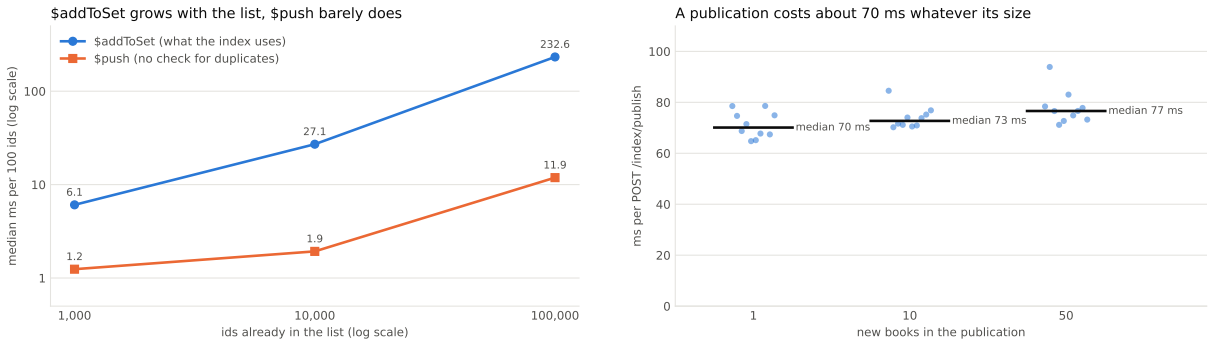



Figure 6: Left: median time to add 100 ids to one posting list. Right: each `POST /index/publish` of 1, 10 and 50 new books on an index of 10,000 books, 10 times each, and the median.

**Publication.** `POST /index/publish` on an index of 10,000 books takes a median of 70, 73 and 77 ms for 1, 10 and 50 new books (Figure 6). The first publication, with all 10,000 books, took 459 ms.

### 7.2.3 Analysis

- **Indexing is limited by MongoDB, not by its own CPU.** From 1 to 8 threads the rate improves by 2 %, within the noise, while the CPU part alone doubles. Indexing uses less than a fifth of one core. MongoDB uses about one core of its two for the whole run, because a single writer sends one bulk write at a time: the work of MongoDB is one serial stream.
- **The rate falls as the index grows, and `$addToSet` explains it.** At 10,000 books a batch of 100 takes about 33 s, at 30,000 about 90 s. The probe shows why: `$addToSet` compares each new id with the whole list, so its cost grows linearly with the list, 14 times that of `$push` at 10,000 ids and 20 times at 100,000. The frequent terms hold almost one id per book, so every batch pays for the size of the index. The two scales agree where they

overlap: books 9,000 to 12,000 of the 30,000 run went at 2.9 books/s, and the last tenth of the 10,000 runs at 3.2.

- **Publishing is cheap and its cost is fixed.** From 1 to 50 new books the median grows by 7 ms: the round trips of its steps dominate, not the books. The Control can therefore publish often, every 30 s or every 100 books, without loading MongoDB.
- **Saturation point.** On this host and these limits, indexing does not saturate a resource of its own: it is bound by the time MongoDB takes per update, which grows with the index. Adding threads or replicas to Indexing changes nothing; changing the write (Section 6.2) or giving MongoDB more cache and CPU would.

**Limits of these measures.**

- Everything runs on one machine: the driver, `docker stats` and the services share its CPUs.
- The WiredTiger cache (0.25 GB) is smaller than the index from 10,000 books on. The probe isolates `$addToSet` on one document, but in the full runs part of the slowdown may also come from the cache.
- MongoDB is not restarted between runs, only its database is deleted, so its peak memory grows along the run whatever the level.
- With 200 books only 1 or 2 samples of the monitor fall inside the run, so its CPU figures are not meaningful.
- 100,000 books were not measured: even at the rate of the last books of the 30,000 run, about one book per second and still falling, the 70,000 more books would take over 19 hours.
- Indexing replicas were not measured for throughput: by design only one writes. Their failover was measured (Section 5.2).

## 7.3 Search

This part covers Search; Section 7.4 consolidates it with the other services. The scripts and the instructions to repeat every measure are in `benchmarks/search/`.

### 7.3.1 Reference machine

All the valid measures come from one machine, idle during the run. `host.txt` records each run.

| | |
|---|---|
| CPU | Intel Core i7-9750H, 2.60 GHz: 6 cores, 12 logical CPUs |
| Memory and storage | 16 GB, NVMe SSD |
| Operating system | Ubuntu 24.04.5, Linux 7.0 |
| JVM | OpenJDK 21.0.12, heap fixed at 4 GB for JMH |
| Power | Plugged in, performance energy profile |
| Docker | 28.3.3, with the 12 CPUs and 16 GB of the host |
| Code | commit `5e85611` for the load tests, `ba4f120` for JMH and the test with the pipeline |

Table 20: Reference machine of the benchmarks.

**Two runs of JMH.** During the first run (7 November) a loop of `docker stats` left over from an earlier test was running, so the machine was not completely idle, as §6.3 of the assignment asks. JMH was therefore measured again on 8 November without it; this section uses that second run, and the first one is kept in `results/jmh-previous`. Intersection and filters agree between both

runs within 5 % in most configurations (the largest difference is 19 %). Pagination and threads, the last benchmarks of the run, came out slower than in the first run, by up to 58 %, with errors of up to 15 %: the same search (`common`, 100,000 books, first page of 20) took 4.92 ms in the intersection benchmark and 7.02 ms in the pagination benchmark a quarter of an hour later, while the first run gave 4.84 ms for both. Nothing else heavy ran on the machine at that time, and its kernel had logged about 11 hours of thermal throttling since it booted, so the most likely cause is that the CPU heated up and lowered its frequency at the end of the run; we could not prove it. The conclusions below compare configurations measured together, and they are the same in both runs; the absolute times of pagination and threads are the least reliable numbers of this section.

### 7.3.2 Microbenchmarks with JMH

The JMH classes are in `search-service/src/test/java/.../benchmark`, so they are not part of the service jar, and they run directly on the host. They measure `SearchEngine` on a synthetic index held in memory, without MongoDB or HTTP, with 200, 10,000 and 100,000 books. Each configuration runs 5 warm-up and 10 measurement iterations of 1 s in 2 separate JVMs with a fixed heap of 4 GB, and each scenario compares its result with an independent calculation before measuring.

| **Benchmark** | **What changes** |
|---|---|
| Intersection | Number of books and of terms in the query |
| Filters | Number of books and which filter |
| Pagination | Number of books, position of the page (0, 50 and 99 %) and its size (20 and 100) |
| Threads | 1, 2, 4 and 8 threads on the same intersection, with 100,000 books |

Table 21: JMH experiments.

**Limitation.** Each term is assigned to books at random and independently, while real words tend to appear together. These results compare the cost of each step; they do not predict the response time of real queries.

In the synthetic index, `common` is in 50 % of the books, `frequent` in 30 %, `medium` in 10 % and `rare` in 1 %, so the number of matches of each query is known in advance. Every value below is the mean time of one search, with its 99.9 % error in the CSV files.

| **Query** | **Matches** | **200 books** | **10,000** | **100,000** |
|---|---|---|---|---|
| `common` | 50 % | 2.6 µs | 373 µs | 4.92 ms |
| `common frequent` | 15 % | 1.4 µs | 287 µs | 3.11 ms |
| `common frequent medium` | 1.5 % | 0.8 µs | 79 µs | 0.84 ms |
| `common rare` | 0.5 % | 0.3 µs | 4.4 µs | 0.16 ms |

Table 22: JMH: intersection without filters, first page of 20, one thread.

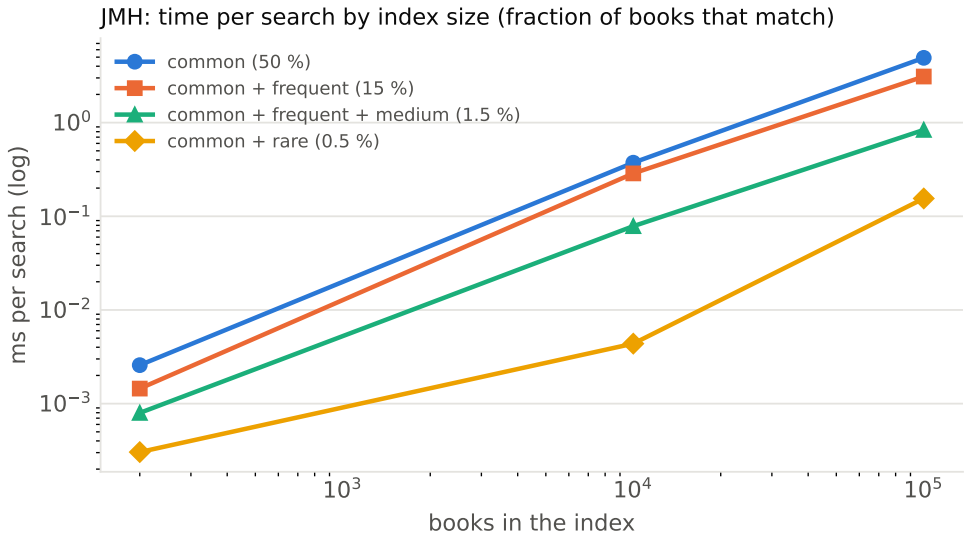



Figure 7: Time per search against the size of the index, both axes logarithmic.

**What the intersection shows.**

- **The cost follows the number of matches, not only the length of the lists.** With 100,000 books the four queries all read the list of `common`, 50,000 ids, yet they range from 4.92 ms to 0.16 ms. A rarer term makes the shortest list shorter, so the intersection walks fewer ids, and fewer matches are then checked against the snapshot, filtered and sorted.
- **Ten times more books cost more than ten times as much at the top of the scale.** From 10,000 to 100,000 books, `common` goes from 373 µs to 4.92 ms, 13.2 times. Sorting every match grows faster than linearly, and the larger lists no longer fit in the CPU caches. These are our explanations; they were not measured separately.

**Filters.** With 100,000 books and the query `common`, no filter takes 5.37 ms. The author filter adds 17 %, because it compares strings ignoring case, and all of them together 14 %; the language filter stays within the error. The year filter takes 15 % less, because it leaves fewer books to sort. The errors here are 10–14 %, so only the author and year filters differ clearly from no filter. Filters run in memory on the metadata of the snapshot, after the intersection, so they never add a query to MongoDB.

**Pagination.** The position of the page does not change the cost: with 100,000 books, the first page, the middle one and one near the end take 7.02, 7.63 and 7.59 ms, within their error of about 1 ms, and pages of 100 results take the same (7.36–7.45 ms). The first run shows the same with lower times (4.84–4.87 ms). Every search does all the work (intersection, filters, ordering of every match) before cutting the page, and the cut itself costs almost nothing. The other side is that the first page is not cheaper either: a user who only looks at 20 results pays for all of them. Caching the result of a query until the next version is published would avoid it, because a snapshot never changes.

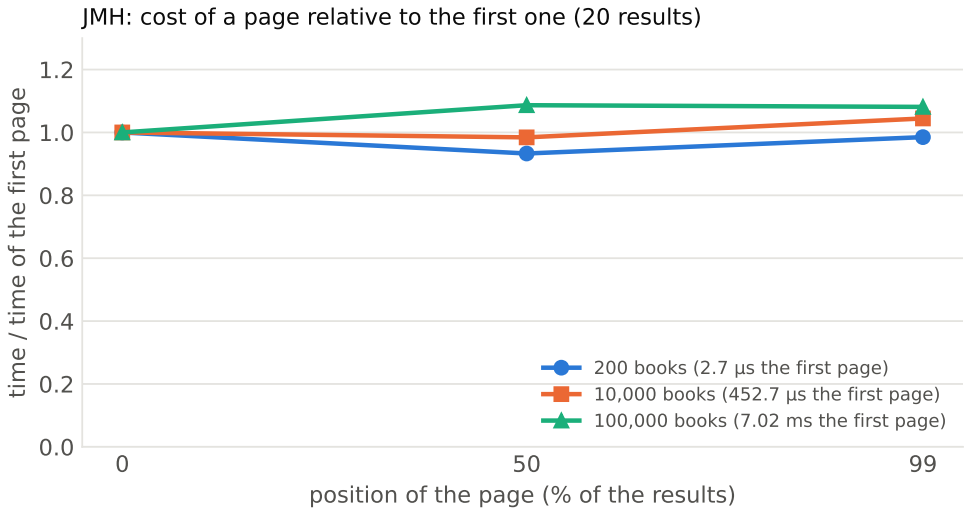



Figure 8: Time of a page relative to the first page, pages of 20 results.

**Threads.** `SearchEngine` has no state of its own and the snapshot is immutable, so several threads search at once without locks. With 100,000 books and `common frequent`, 2 threads complete 1.9 times as many searches per second as one, 4 threads 2.9 times and 8 threads 5.1 times (1.9, 3.6 and 5.7 in the first run). Each search becomes slower as threads are added, from 3.59 ms with one to 5.63 ms with eight. The threads do not wait for each other, but they share the CPU caches and the memory bandwidth, and 8 threads exceed the 6 physical cores of the machine.

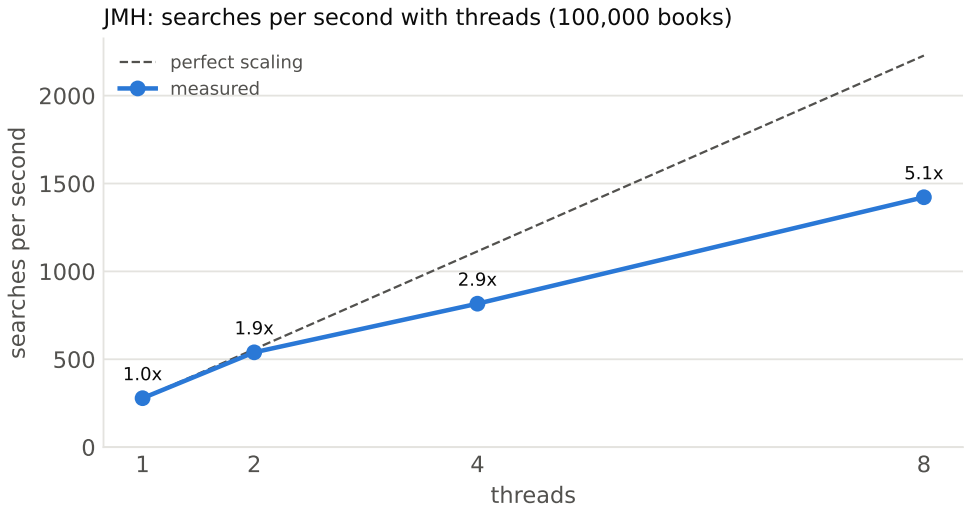



Figure 9: Searches per second with 1 to 8 threads, against perfect scaling.

This is the cost of the code alone. A real search adds MongoDB, which returns the posting lists, HTTP and nginx: the load tests measure them.

### 7.3.3 Load tests

`LoadGenerator` sends `GET /search` through nginx with `shared/queries.txt`, and checks each answer against the total and version it obtained first: more replicas must not change the results.

Before measuring, the script runs `validate-api.sh`. Each configuration is repeated 3 times with 15 s of warm-up and 30 s of measure, and one factor changes at a time:

| Experiment | What changes | Fixed |
|---|---|---|
| Clients | 1, 8, 32, 128, 256 clients with 1, 2 and 4 replicas | 250 threads, pool of 50, nginx with 2 CPUs |
| Threads | 16 and 64 Jetty threads, with the same clients | 1 replica, pool of 50, nginx with 2 CPUs |
| Pool | 10 and 100 MongoDB connections, with 32, 128 and 256 clients | 1 replica, 250 threads, nginx with 2 CPUs |
| nginx | 0.5 CPUs, the limit of the deployment, with 32, 128 and 256 clients | 1 replica, 250 threads, pool of 50 |
| Index size | The clients experiment with 197 real books and with 10,000 synthetic ones | Same as the clients experiment |

Table 23: Load experiments. Client concurrency, threads per process and replicas are kept apart.

**Why nginx has 2 CPUs here.** Short trial runs showed that nginx, with the 0.5 CPUs of the deployment, saturated before Search and hid the effect of everything else. The experiments therefore give it 2 CPUs and measure the 0.5 limit separately.

**Limitations.**

- The generator runs on the same machine as the system, so it competes with it for CPU.
- It is a closed loop: under saturation, clients wait instead of queuing more requests, so latencies are lower than with requests arriving at a fixed rate.
- With 4 replicas the limits of the containers add up to 14 CPUs, which may exceed the host. Then the limits no longer bound each container, and the replicas compete for the same cores.

**The indexes.** The 15 books of the Stage 1 sample; 197 of the 200 real books of `shared/book_ids.txt` (3 could not be ingested from Project Gutenberg); and 10,000 synthetic books (DR-16), indexed by the whole pipeline (Control, Ingestion and Indexing) in 2 h 49 min before the measures. Search and MongoDB have 2 CPUs each (each Search replica has its own 2), nginx has 2 except in the nginx experiment, and the three containers have 1 GB of memory, nginx 128 MB.

**Results** The 192 runs sent 29.6 million requests. **No answer had a wrong result**, with up to 4 replicas. There were **3 HTTP errors**, all in the most overloaded configuration (10,000 books, one replica, 256 clients: 3 of 27,784 requests, with a slowest answer of 5.5 s); the generator does not keep their status code. No request timed out or failed to connect.

| Books | Replicas | 1 client req/s | Peak req/s | p95, 32 cl. ms | p95, 256 cl. ms | CPU at its limit |
|---|---|---|---|---|---|---|
| 15 | 1 | 2,363 | 7,579 | 7.7 | 91.7 | Search and MongoDB |
| 15 | 2 | 2,182 | 7,652 | 7.1 | 88.8 | MongoDB |
| 15 | 4 | 1,950 | 7,391 | 8.3 | 79.2 | MongoDB |
| 197 | 1 | 2,040 | 5,932 | 30.2 | 96.3 | Search |
| 197 | 2 | 1,914 | 7,347 | 8.5 | 99.1 | MongoDB |
| 197 | 4 | 1,703 | 7,071 | 9.8 | 80.3 | MongoDB |
| 10,000 | 1 | 532 | 1,036 | 87.7 | 984.6 | Search |
| 10,000 | 2 | 513 | 1,729 | 69.9 | 792.3 | Search |
| 10,000 | 4 | 495 | 2,612 | 31.5 | 515.5 | Search, and the host |

Table 24: Clients experiment: throughput with one client and at its peak, p95 with 32 and 256 clients, and the container whose CPU reaches its limit (mean of 3 repetitions; standard deviations are in `tables.md`, at most 3.2 % of the throughput).




Figure 10: Throughput against clients for 1, 2 and 4 replicas, for each index.




Figure 11: p95 latency against clients (logarithmic scale).

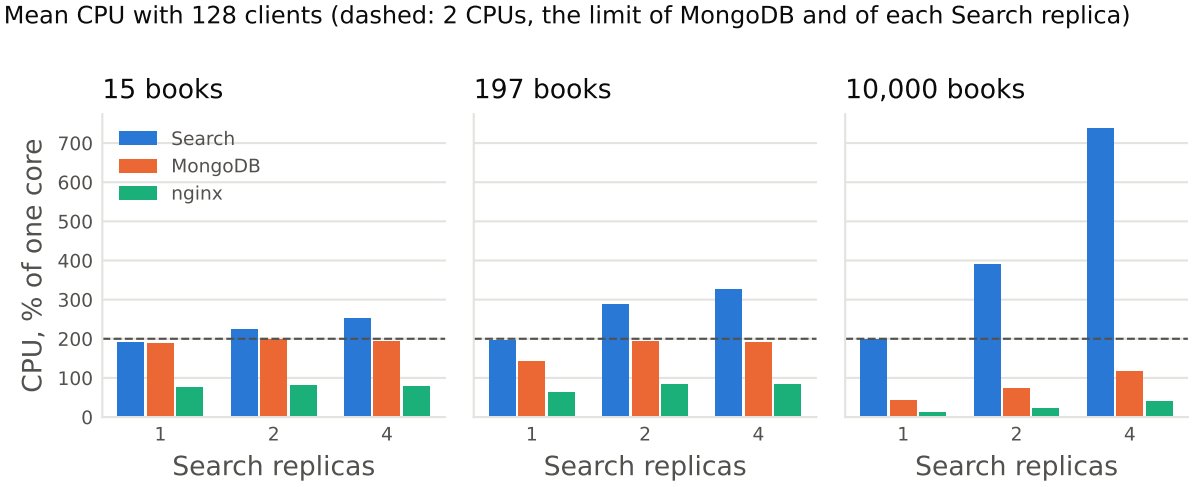



Figure 12: Mean CPU per container with 128 clients. Search adds its replicas, so its limit is 200 % per replica.

| Configuration (15 books, 1 replica) | Clients | req/s | p50 ms | p95 ms | p99 ms |
|---|---|---|---|---|---|
| 250 Jetty threads, pool of 50 (reference) | 256 | 7,548 | 23.9 | 91.7 | 179.6 |
| 16 Jetty threads | 256 | 7,841 | 27.2 | 66.8 | 74.4 |
| 64 Jetty threads | 256 | 7,802 | 27.1 | 67.2 | 77.0 |
| Pool of 10 MongoDB connections | 128 | 6,887 | 4.3 | 78.6 | 119.4 |
| Pool of 10 MongoDB connections | 256 | 6,805 | 3.9 | 166.4 | 280.3 |
| Pool of 100 MongoDB connections | 256 | 7,634 | 25.7 | 78.6 | 140.4 |
| nginx with 0.5 CPUs (the deployment) | 32 | 5,331 | 2.3 | 58.5 | 65.3 |
| nginx with 0.5 CPUs (the deployment) | 256 | 5,607 | 22.5 | 93.8 | 99.7 |

Table 25: Threads, pool and nginx: one factor changed against the reference configuration.

### 7.3.4 Analysis

**Where it saturates.** The throughput stops growing between 8 and 32 clients in every configuration. From there on, more clients only wait longer: the generator is a closed loop, so the mean latency is the number of clients divided by the throughput, as Little's law says (with 15 books, one replica and 256 clients, 256 / 7,548 req/s = 33.9 ms, and the measured mean is 33.9 ms). p95 and p99 grow in the same proportion: with 15 books and one replica, p95 goes from 1.4 ms with 8 clients to 91.7 ms with 256, for the same throughput.

**What limits it depends on the size of the index** (Figure 12 and Table 24).

- **15 books: MongoDB.** A search over a tiny index costs Search almost nothing, but each one still asks MongoDB for the posting list of every term. With one replica, Search (192 %) and MongoDB (175–190 %) are both at their 2 CPUs. More replicas give Search spare CPU (it uses 239–279 % of the 400–800 % it has), but MongoDB stays at 193–200 %, and the throughput stays at about 7,500 req/s.
- **197 books: Search, then MongoDB.** Longer posting lists make the intersection more expensive, and one replica saturates first (198 %, while MongoDB uses 148 %): 5,900 req/s. A second replica gives 24 % more (7,350 req/s), until MongoDB reaches its limit (184–195 %); a fourth one adds nothing.
- **10,000 books: Search.** Each search has to intersect, filter and order almost the whole index (see the limitations below), so Search spends far more CPU per request: one replica gives 1,036 req/s at its limit (197–200 %) while MongoDB uses only 42 %. Here replicas do help: 2 replicas give 1.7 times as many requests and 4 replicas 2.5 times. Four replicas do not reach 4 times because the host runs out: the containers use 9.3 to 9.5 of its 12 logical CPUs (6 physical cores), and the generator shares them.

So a larger index does **not** move the bottleneck to MongoDB; it moves it the other way, to the CPU of Search. MongoDB returns longer lists, but Search does more work with each of them. This is what the JMH results predicted: the cost of a search grows with the number of books that match.

**Replicas.** They increase the throughput only while Search is the bottleneck, and only up to the next one (MongoDB with small indexes, the host with 4 replicas and 10,000 books). They always help the tail: with 256 clients, p99 falls from 180 to 98 ms with 15 books, from 250 to 104 ms with 197 and from 1,833 to 1,272 ms with 10,000 when going from 1 to 4 replicas, because each JVM has fewer requests waiting behind a slow one or a garbage collection. With a single client, more replicas are slightly slower (2,363 to 1,950 req/s with 15 books, about 0.1 ms more per request); each request goes to one replica in any case, and nginx spreads consecutive requests over JVMs that are each less warm. We did not study it further.

**Jetty threads.** 250 threads give no more throughput than 16 or 64, and a worse tail (Table 25): with 256 clients, 16 threads give 4 % more requests and a p99 of 74 ms instead of 180 ms. With 2 CPUs, the extra threads only compete for them and are preempted in the middle of a request; with few threads, the requests wait in Jetty's queue and are then served without interruption. A value between 16 and 64 would be a better default than 250.

**The MongoDB pool.** The pool of 50 connections, which was an estimate, is enough. A pool of 10 is too small with 128 clients or more: the throughput falls 9–10 %, p95 rises 1.5 to 1.8 times, and p50 *falls* to about 4 ms. The requests that get a connection are served quickly, while the rest wait for one; MongoDB is then less busy (166–167 %). A pool of 100 gives the same as 50 (within about 1 %), because MongoDB is already at its CPU limit.

**nginx in the deployment.** With the 0.5 CPUs of the deployment, nginx is the first bottleneck: it reaches its limit (49–50 %) and caps the system at 5,300–5,600 req/s, 26–29 % less than with

2 CPUs, while Search and MongoDB stay at 114–126 % of their 200 %, with a p95 of 58.5 ms already at 32 clients (7.7 ms with 2 CPUs). That is why the other experiments give it 2 CPUs. A deployment that must serve more than about 5,000 searches per second should raise `NGINX_CPUS`.

**Memory.** No container was short of memory except MongoDB with 10,000 books, which reached 1,024 MB, its 1 GB limit (Search used at most 372 MB). It did not limit these tests, where MongoDB used only 42–121 % of its CPU, but its cache would be the next limit with a larger index.

**Limitations of these results.**

- The synthetic books spread the words at random, so a common word is in almost every book (*whale* is in 293 of 300). With 10,000 books almost every query matches almost the whole index, which is the worst case for Search: real books would give shorter results and higher throughput.
- The generator shares the machine with the system and competes with it for CPU, most of all with 4 replicas.
- `containers.csv` takes one sample about every 2 s, so short peaks of CPU do not appear in the means.
- 100,000 books were not tested under load: the pipeline indexes about one book per second on this machine, so building that index would take more than a day. JMH covers 100,000 books without HTTP or MongoDB.

### 7.3.5 Search while the pipeline works

`run-mixed.sh` measures how Search and the pipeline affect each other (§6.4 of the assignment). It builds an index of 2,000 synthetic books with the whole system, one replica of each service, and then measures Search with 32 and 128 clients twice: *idle*, and *mixed*, while the Control, Ingestion and Indexing process 4,000 more books. It also measures the pipeline alone and under the load of Search. Each phase is repeated 3 times, 60 s each.

| Phase | Clients | req/s | p50 ms | p95 ms | p99 ms |
|---|---|---|---|---|---|
| Idle | 32 | 2,903 | 4.6 | 63.8 | 68.4 |
| Mixed | 32 | 2,686 | 5.3 | 61.1 | 67.6 |
| Idle | 128 | 3,017 | 18.5 | 108.7 | 200.9 |
| Mixed | 128 | 2,594 | 24.0 | 160.4 | 249.8 |

Table 26: Search idle and while the pipeline works (mean of 3 repetitions; 2,000 books, one replica of each service). The pipeline indexes 2.42 ± 0.23 books/s alone and 0.91 ± 0.04 books/s under the load of Search.

**They meet in MongoDB.** With the pipeline idle, Search is at its 2 CPUs (201–202 %) and MongoDB uses 80 %: with 2,000 synthetic books the queries are long, as in the index of 10,000. When the pipeline works, MongoDB rises to 191–192 %, its limit, because of the writes of Indexing; Indexing itself uses 3 % of its CPU, because it waits for MongoDB. Search keeps its CPU, but each query waits longer for its posting lists: with 128 clients it serves 14 % fewer requests and its p95 rises by 48 %; with 32 clients, 7 % fewer and the same p95. The pipeline suffers more: it falls to 38 % of its rate, because its own bottleneck, MongoDB, is now shared. Both kept working and no answer was wrong, so the services interfere only through the capacity of MongoDB. On one host this is the main limit of the system; a MongoDB that serves the reads of Search apart from the writes of Indexing (a replica set) would remove most of it.

## 7.4 Consolidation

This part puts the measures of the three services side by side: one host, the configurations measured, and where each part of the system saturates.

### 7.4.1 The host

Every valid measure ran on one machine, recorded by the `host.txt` of each run:

| | |
|---|---|
| CPU | Intel Core i7-9750H, 2.60 GHz: 6 cores, 12 logical CPUs |
| Memory and storage | 16 GB; NVMe SSD |
| Operating system | Ubuntu 24.04.5, Linux 7.0 |
| Docker | 28.3.3, sees the 12 CPUs and 16 GB |
| JVM | OpenJDK 21.0.12 for JMH and the drivers; the images run Java 17 |
| Power | Plugged in, performance energy profile; under sustained load the CPU is throttled by its temperature |

Table 27: Reference machine of all the benchmarks.

The deployment limits each container (`.env`): MongoDB 2 CPUs and 1 GB with a 0.25 GB cache; Indexing 2 CPUs and 1 GB; Search 2 CPUs per replica; Ingestion 1 CPU and 512 MB; each Hazelcast member 1 CPU and 768 MB; the Control and nginx 0.5 CPUs. The load experiments of Search give nginx 2 CPUs, so that it does not hide the rest, and measure its 0.5 CPUs apart. The limits together exceed the 12 CPUs of the host only in the Search experiments with 4 replicas (14 CPUs), where the replicas compete for the same cores.

### 7.4.2 What was measured

| Experiment | Scope | Replicas | Threads and clients |
|---|---|---|---|
| Ingestion JMH | component, host | – | 1, 2, 4, 8 workers |
| Indexing JMH | component, host | – | 1, 2, 4, 8 parser threads |
| Search JMH | component, host | – | 1, 2, 4, 8 threads |
| Indexing, threads | Indexing + MongoDB + Hazelcast, Docker | 1 Indexing | 1, 2, 4, 8 parser threads; 2,000 books |
| Indexing, scale | same | 1 Indexing | 4 threads; 200, 10,000, 30,000 books |
| Indexing, publication | same | 1 Indexing | 1, 10, 50 new books on 10,000 |
| Search, load | Search + MongoDB + nginx, Docker | 1, 2, 4 Search | 1 to 256 closed-loop clients; 250 Jetty threads (also 16 and 64); pool of 50 (also 10 and 100); nginx with 2 CPUs (also 0.5); index of 15, 197 and 10,000 books |
| Search with the pipeline | complete pipeline, Docker | 1 of each | 32 and 128 clients on 2,000 books while the pipeline processes new ones |

Table 28: The experiments, each labelled with what it measures. Every configuration was repeated 3 times, except the 30,000 books of Indexing (once). Client concurrency, replicas and threads are kept apart.

### 7.4.3 Bottlenecks and scaling trends

| Service | Measured rate | What limits it |
|---|---|---|
| Ingestion | 473 $\mu$s per book with 1 worker, 155 with 8 (local source, JMH) | The coordination: claim and complete cost 1.6 ms per book with 1 thread and 0.36 with 8, more than the ingestion itself. With Project Gutenberg the download is far longer than both. |
| Indexing | 15 books/s on 2,000 books; 5.5 on 10,000; 2.0 on 30,000 | MongoDB: the cost of `$addToSet` grows with each posting list, and one writer sends one stream of writes. Threads and replicas of Indexing do not change it (Section 7.2). |
| Search | About 7,500 req/s on 15 books whatever the replicas; on 10,000 books 1,036 req/s with 1 replica and 2,612 with 4 | On a small index MongoDB is the limit: it reaches its 2 CPUs while the replicas of Search have CPU to spare. On 10,000 books each query matches most of the index, the CPU of Search is the limit, and replicas help: 4 give 2.5 times the peak of one, less than 4 because the host runs out of CPU (Section 7.3). |
| The whole system | with the pipeline running, Search 2,600 req/s and Indexing 0.91 books/s at the same time | MongoDB is shared: the writes of Indexing take it from 80 to 191 % of its 2 CPUs. Indexing drops from 2.42 to 0.91 books/s while Search is under load, and the p95 of Search with 128 clients rises from 109 to 160 ms. |

Table 29: Where each part of the system saturates on the reference machine.

**Threads and replicas.** Threads help where the work is CPU inside one process: the workers of Ingestion and the parsers of Indexing in JMH, and the replicas of Search when queries are long. They do not help where the limit is a shared resource: the queue of Hazelcast for Ingestion with small books, MongoDB for Indexing, and MongoDB again for Search on a small index. Indexing replicas are standbys by design, so their effect is failover, not throughput.

**What was not measured.** The full pipeline with several Ingestion replicas and Project Gutenberg as the source (the measures use the local source, so that the network does not dominate); 100,000 books, because indexing them at the measured rate would take most of a day; and two machines, which is Stage 3.

# 8. Conclusions and evolution to Stage 3

## 8.1 Strengths and weaknesses of the coordination

**Strengths.** The coordination is explicit and checked: every transition is one method of one library, atomic, and tested against real Hazelcast members. A book never changes state before its evidence is persistent, so a crash at any point leaves a book that is either redone or recovered; the failure tests check it end to end (Table 12). Replicas of Ingestion and Search are added with `--scale` and nothing else, because nobody addresses a replica: the workers pull from shared queues, and nginx spreads the queries. Losing the whole Hazelcast cluster costs a recovery, not data.

**Weaknesses.** Hazelcast is in memory, so the books without evidence (`PENDING`, or `INGESTING` without `_READY`) must be sent again after the cluster is lost. The lock of the index writer has no fencing token. The leases compare the clocks of different processes. The datalake depends on a local file system and its atomic moves. Each book costs two transactions, and both queues are single partitions that every claim goes through.

**What the measures say.** The JMH of Ingestion (Section 7.1) show that coordinating a book through Hazelcast costs more than ingesting it from the local source (1.6 ms against 0.47 ms with one worker), and that more workers hide that cost well (0.36 ms per book with 8). Even so, Ingestion runs two orders of magnitude faster than Indexing, so it does not limit the pipeline on this machine. The behaviour with several replicas and with the real download was not measured.

## 8.2 What changes on the Stage 3 cluster

The services keep their code; what they assume about the machine does not hold on a cluster. Table 30 lists each local assumption and what replaces it.

| Local assumption | Why it breaks on a cluster | Stage 3 |
|---|---|---|
| Two Hazelcast members on one host | They share its failures; a backup on the same machine protects from nothing | One member per machine (or embedded in the services), discovery by the platform instead of a fixed list, and backups placed on another machine (partition groups per host or zone) |
| `IMap.lock` for the single writer | In a network partition each half of the cluster can grant it (split-brain), and it has no fencing token | A CP `FencedLock` (Enterprise, three or more CP members), or a lease document in MongoDB taken with a conditional update whose version is the fencing token for every write; a split-brain protection quorum on the coordination structures |
| Leases on one clock | Clocks of different machines drift; a lease could expire early or late | NTP on every node and a margin above the expected skew, or the time of the member that owns the entry |
| Datalake on a local file system, `_READY` after atomic moves | An atomic move exists only inside one file system; Indexing on another machine cannot read the folder | Shared storage (an object store such as S3 or MinIO, or HDFS): write the book, then the marker object; readiness is the marker object |
| Recovery lists the datalake | Listing every range folder grows with the number of books, and gets slower over the network | A persistent log of ready books (a MongoDB collection written with the marker), read like `index_log` |
| One Control | Its absence stops leases and publications; on a cluster, nodes come and go | Two or more instances with a leader election (the same fenced lease as the writer); the others wait as standbys |
| Queues of one partition | Every claim of every node goes to one member | Several queues by range of ids, each taken by some workers, or a partitioned log with acknowledgements (Kafka) if the work grows beyond what one member serves |
| Services without authentication on `127.0.0.1` | On a network of machines anyone could reach the Control or Hazelcast | TLS and authentication on the Hazelcast clients and on the internal APIs; only the load balancer exposed |
| Docker network on one host | Containers on different machines do not share it | An overlay network or an orchestrator (Kubernetes, Docker Swarm) with service discovery; nginx replaced by its load balancer |

Table 30: The local assumptions of the coordination and their Stage 3 replacement.

**What does not change.** The contract itself: the states, the transitions, the principle that evidence comes first, the leases and the recovery are the same on one machine and on many, and the services reach Hazelcast through the same library. That was the reason to choose it (Table 11): for Stage 3 the work is in the configuration and in the storage, not in the services.

**Distribution of work.** On a cluster the replicas of Ingestion run on several machines and keep pulling from the same queues, so the distribution of books needs no change. What needs measuring again is where the first bottleneck moves once each replica has a machine of its own: the queues of Hazelcast, the shared storage, or the single index writer. On this machine the first limit is already MongoDB, shared by the writer of Indexing and every replica of Search (Section 7.4); for Ingestion it is the round trips to Hazelcast. On a cluster both move: MongoDB needs replication or sharding, and the coordination pays a real network on every transaction, which makes batching the claims worth considering.